

Дизайн и анализ на алгоритми

план на упражненията

КН 2.1, летен семестър 2024/2025

Калоян Цветков
kaloyants25@gmail.com

ФМИ, СУ

2.4

Съдържание

1	Въведение	4
1.1	Алгоритъм	4
1.2	Изчислителен модел	5
1.2.1	Въвеждане на RAM модела	5
1.2.2	Представяне на данни в паметта	7
1.2.3	Операции за константно време в RAM модела	7
1.2.4	Псевдокод	7
1.3	Първи пример	8
1.4	Коректност на алгоритъм	9
1.4.1	Итеративен подход	9
1.4.2	Рекурсивен подход	9
1.5	Времева сложност	9
1.6	Асимптотика	9
1.6.1	Основни свойства	10
2	Линейни обхождания на масив	11
2.1	Сложност по памет	11
3	Сортиране	17
4	Разделяй и владей	25
5	Алчни алгоритми	34
6	Графи	40
7	Динамично програмиране	50
8	P, NP, NP-HARD, NP-COMPLETE	57
9	Задачи, неразгледани на упражнение	60

Списък с определения, задачи, важни твърдения и допълнения

1.1	Определение – Размер на входа	5
1.2	Определение – Машина с произволен достъп или RAM	5
1.3	Определение – Елементарна операция в RAM	5
1.4	Определение – Цена на елементарна операция в RAM	6
1.1	Задача – GCD	8
1.5	Определение – Асимптотично сравнение - $\preceq$	9
1.6	Определение – <i>Big O</i>	9
1.7	Определение – <i>Big Ω</i>	9
1.8	Определение – <i>Big Θ</i>	10
2.1	Задача – MAXIMUM SUBARRAY	11
2.2	Задача – DISTANT PAIR	12
2.3	Задача – SIEVE	13
2.7	Твърдение – техника за работа със сума от монотонна непрекъсната функция	14
2.1	<i>Допълнение</i> – ($O(n \log(\log n))$) е по-добра оценка за SIEVE)	14
2.4	Задача – Линейно решето	15
3.1	Задача – 2-SUM	17
3.2	Задача – 3-SUM	18
3.3	Задача – 4-SUM	18
3.4	Задача – MINIMUM ABSOLUTE SUBARRAY	18
3.5	Задача – TRIANGLES	19
3.6	Задача – GROUPING	20
3.7	Задача – Задача 1, писмен изпит, редовна сесия, 2023г.	21
3.1	Определение – Редукция на изчислителни задачи	23
3.8	Задача – 3-SUM-TARGET	23
3.9	Задача – 3-COLLINEAR	24
3.10	Задача – CONCURRENT LINES	24
3.1	<i>Допълнение</i> – Редукция към CONCURRENT LINES	24
4.1	Задача – FLIP	25
4.2	Задача – Домашно 1, писмен изпит, редовна сесия, 2023г.	26
4.5	Твърдение – Частни случаи на уравнение, породено от <i>Разделяй и владей</i>	27
4.3	Задача – EXPONENT	27
4.4	Задача – CLOSEST PAIR	29
4.5	Задача – k -ти по големина елемент в два сортирани масива	32
4.10	Твърдение – долна граница за CLOSEST PAIR	32
4.6	Задача – 3-EQUAL	33
4.7	Задача – MODE	33
5.1	Задача – MAXIMUM STOCKS	34
5.2	Задача – MAKE CHANGE	34
5.3	Задача – INTERVAL SCHEDULING , Домашно 1, 2023г.	35
5.4	Задача – MINIMUM HALLS	36
5.5	Задача – MINIMUM AVERAGE COMPLETION TIME	37
5.6	Задача – MINIMUM SCHEDULING PENALTY (Матроиди)	38
6.1	Задача – TRIANGLE	40
6.2	Задача – Хамилтонов път в турнир	42
6.3	Задача – Второ най-леко покриващо дърво	43
6.4	Задача – LONGEST TREE PATH	44
6.5	Задача – MINIMUM TURNS	45
6.6	Задача – k -Clustering	46
6.7	Задача – DIRECTED WIDEST PATH	48
6.8	Задача – UNDIRECTED WIDEST PATH	49

6.8	Твърдение – Най-тесни пътища	49
7.1	Задача – ODD COST DAG	50
7.2	Задача – SHORTEST PATHS COUNT	50
7.3	Задача – MAKE CHANGE	50
7.4	Задача – Брой думи с дължина n в регулярен език	51
7.5	Задача – LONGEST COMMON SUBSEQUENCE	52
7.6	Задача – LCS PERMUTATIONS	53
7.4	Твърдение – редукция на LIS към LCS	53
7.7	Задача – Контролно 2, 2022/2023г.	53
7.8	Задача – MINIMUM MEAN WEIGHT CYCLE	54
7.9	Задача – BITONIC EUCLIDEAN TRAVELING SALESMAN	55
8.1	Определение – класът проблем P	57
8.2	Определение – класът проблеми NP	57
8.3	Определение – Редукция за полиномиално време	57
8.4	Определение – класът проблеми NP-HARD	57
8.5	Определение – класът проблеми NP-COMPLETE	57
8.2	Твърдение – TRAVELING SALESMAN е NP-COMPLETE	58
8.1	Задача – INTEGER (LINEAR) PROGRAMMING	58
8.3	Твърдение – IP ∈ NP-HARD (редукция от 3SAT)	58
8.4	Твърдение – IP ∈ NP-HARD (редукция от VERTEX COVER)	59
8.1	Допълнение – Някои редукции за полиномиално време	59
9.1	Задача – HOLE	60
9.2	Задача – Домашно 1, Контролно 1, 2023г.	60
9.3	Задача – Алгоритъм на <i>Boyer – Moore</i>	60
9.4	Задача – MAX CLUSTER	60
9.5	Задача – INTERSECTING SEGMENTS	60
9.6	Задача – LOCAL MIN	61
9.7	Задача – TYPESETTING	61
9.8	Задача – STABLE MATCHING	61
9.9	Задача – CAMPAIGNING	62
9.10	Задача – TRAINING HILLS	62

1 Въведение

1.1 Алгоритъм

Също както за *множество*, така и за *алгоритъм* няма общоприета формална дефиниция. В този курс *алгоритъм* ще интерпретираме така:

- Алгоритъм е крайна редица от операции, която решава дадена задача.
Разглеждаме го като реализация на тотална функция $A : Input \mapsto Output$, където *Input* и *Output* са крайни редици от числа и масиви.

Потенциални въпроси:

- "Защо редицата от операциите е крайна?"
Можем да отслабим изискването за крайност и ще получим т.н. *изчислителен метод*. В рамките на курса ще разглеждаме единствено редици с краен брой операции.
- "Какво представлява една *операция*?"
Зависи от *изчислителния модел*. В курса предимно ще се използва RAM моделът. В съответната секция ще бъдат описани позволените в RAM модела *операции*.
- "Какво представлява една *задача* и как тя се решава?"
Става въпрос за *изчислителна задача* - понятие, което има дефиниция. Характеризира се със своите *екземпляри*, на всеки от които съответстват неотрицателен брой *решения*.
Формално, за изчислителна задача може да се счита всяка релация над $\mathbb{N}^*$. Можем да мислим за *Input* и *Output* като описания на *екземпляри* и *решението* съответно (или едно от всички възможни *решения*).
- "Защо *Input* и *Output* са крайни редици и защо в тях участват само числа и масиви?"
Тук отново може да бъде отслабено изискването за крайност, но в рамките на курса няма да разглеждаме задачи с неограничено големи *Input* и *Output*.
Използваме числа и масиви за описание на *Input* и *Output*, понеже с тях могат да се представят математическите обекти, за които ще решаваме задачи.
- "Какво разбираме под 'числа и масиви'?"
Числата могат да са от $\mathbb{N}, \mathbb{Z}$ и $\mathbb{Q}$. При $\mathbb{R}$ възниква въпросът за представянето, чрез апроксимация с крайна точност (тази тема може да бъде засегната по-късно).
Масивите са крайни редици от числа или от други масиви. Множество на масивите - $\mathbb{M}$.
За $\mathbf{I} = \{I_i\}_{i=1}^n, n \in \mathbb{N}^+$ - редица от числа, $\mathbf{I} \in \mathbb{M}$.
За $\mathbf{M} = \{M_i\}_{i=1}^n, n \in \mathbb{N}^+$, за която $M_i \in \mathbb{M}$, $\mathbf{M} \in \mathbb{M}$.

Разговорно ще наричаме *Input* вход на алгоритъма и *Output* изход на алгоритъма. Както входът, така и изходът притежават характеристика *размер* $\in \mathbb{N}^+$, определен от съдържанието на този вход.

Размерът на число $n \in \mathbb{N}$ се дава с $S_{\mathbb{N}} : \mathbb{N} \mapsto \mathbb{N}^+$:

$$S_{\mathbb{N}}(n) = \begin{cases} 1 & \text{ако } n = 0 \\ \lfloor \log_2(n) \rfloor + 1 & \text{иначе} \end{cases}$$

Естествено може да бъде продължена дефиницията за $\mathbb{Z}, \mathbb{Q}$.

Размерът на масив $\mathbf{M} = \{M_i\}_{i=1}^m, m \in \mathbb{N}^+$ се дава с $S_{\mathbb{M}} : \mathbb{M} \mapsto \mathbb{N}^+$ и $S : \mathbb{N} \cup \mathbb{M} \mapsto \mathbb{N}^+$:

$$S_{\mathbb{M}}(\mathbf{M}) = \sum_{i=1}^m S(M_i)$$

$$S(a) = \begin{cases} S_{\mathbb{N}}(a) & \text{ако } a \in \mathbb{N} \\ S_{\mathbb{M}}(a) & \text{иначе} \end{cases}$$

Размерът на редица от числа и масиви $\{E_i\}_{i=1}^n, n \in \mathbb{N}^+$ е сумата от размерите на елементите ѝ.

Определение 1.1 (Размер на входа). *Размер на входа $Input$ наричаме размерът на крайната редица от числа и масиви, които $Input$ представлява.*

1.2 Изчислителен модел

1.2.1 Въвеждане на RAM модела

Машина с произволен достъп или RAM (от английски: Random Access Machine). Това е еквивалентен модел на машините на Тюринг като изразителна способност, но е по-близък до общата представа за модерен компютър.

Определение 1.2 (Машина с произволен достъп или RAM). *Машините с произволен достъп спадат към клас машини с непоследователен достъп до паметта (Random Access Memory или RAM памет). Всяка машина с произволен достъп се състои от памет и програма.*

Паметта на машината е разделена на две части:

- Краен брой регистри $\gamma_0, \gamma_1, \dots, \gamma_n, n \in \mathbb{N}^+$.
- Основна памет, която се състои от безкрайно много клетки номерирани $0, 1, 2, \dots$

Регистри	Памет
регистър γ_0	0
регистър γ_1	1
регистър γ_2	2
...	3
регистър γ_n	...

С $\gamma_k \in \mathbb{Z}, k \in \{0, \dots, n\}$ ще означаваме стойността, която е в регистъра γ_k .

С $\rho(i) \in \mathbb{Z}, i \in \mathbb{N}$ ще означаваме стойността, която е в i -тата клетка на паметта.

Стойностите в регистрите и в паметта могат да имат произволно голям размер (тип данни *integer*).

Програма на машина с произволен достъп е крайна редица от номерирани елементарни инструкции.

Определение 1.3 (Елементарна операция в RAM). За определеност нека означим с α един от регистрите. Без ограничение на общността избираме γ_0 . Регистъра α ще наричаме акумулатор. В него се акумулира резултатът от аритметичните операции. Останалите регистри $\gamma_1, \gamma_2, \dots, \gamma_n$ ще наричаме индексни регистри. Нека $i, j, k \in \mathbb{N}$. По-долу ще използваме:

- *reg*, за да означим произволен регистър от $\alpha, \gamma_1, \dots, \gamma_n$.
- *op*, за да означим операнд от вида $i, \rho(i)$ или *reg*.
- *top*, за да означим модифициран операнд от вида $\rho(i + \gamma_j)$. Стойността на $\rho(i + \gamma_j)$ е в клетката на паметта на позиция $(i + \text{стойността на } \gamma_j)$.

Елементарните операции разделяме на следните категории:

1. Операции за достъп до паметта (четене и писане):

- $reg \leftarrow op$
- $\alpha \leftarrow mop$
- $op \leftarrow reg$
- $mop \leftarrow \alpha$

2. Операции за преход (*jump*):

- $goto\ k$
- $\underline{if\ reg\ \pi\ 0\ then\ goto\ k}$, за $\pi \in \{=, \neq, <, \leq, >, \geq\}$

3. Аритметични операции:

- $\alpha \leftarrow \alpha\ \pi\ mop$, за $\pi \in \{+, -, \times, div, mod\}$

4. Операции с индексните регистри:

- $\gamma_j \leftarrow \gamma_j \pm i$, $1 \leq j \leq n, i \in \mathbb{N}$

Горната дефиниция на RAM позволява да се съхраняват и обработват произволно големи числа, но това не е реалистично предположение. Поради това внимателно ще дефинираме *цената* за изпълнение на елементарните операции.

Цената за изпълнение на елементарна операция се състои от **достъпа до паметта** и **същинската цена за изпълнение**.

Има два основни подхода за определяне на *цената*: UCM (Unit Cost Measure) и LCM (Logarithmic Cost Measure). При UCM се абстрахираме от *размера* на операндите. Това е подходящ подход, при условие че *размерът* на числата, които са в паметта по време на изпълнение на програмата е ограничен отгоре. При LCM взимаме под внимание *размера* на операндите. Използваме дефинираната нагоре функция за *размера* S .

Определение 1.4 (Цена на елементарна операция в RAM). *Определя се според:*

- Цена за достъп до паметта според операнд:

Операнд	UCM	LCM
i	0	0
reg	0	0
$\rho(i)$	1	$S(i)$
$\rho(i + \gamma_j)$	1	$S(i) + S(\gamma_j)$

- Същинска цена за изпълнение:

Операция	UCM	LCM
$reg \leftarrow op$	1	$1 + S(op)$
$\alpha \leftarrow mop$	1	$1 + S(mop)$
$op \leftarrow reg$	1	$1 + S(reg)$
$mop \leftarrow \alpha$	1	$1 + S(\alpha)$
$goto\ k$	1	$1 + S(k)$
$\underline{if\ reg\ \pi\ 0\ then\ goto\ k}$	1	$1 + S(k)$
$\alpha \leftarrow \alpha\ \pi\ mop$	1	$1 + S(\alpha) + S(mop)$
$\gamma_j \leftarrow \gamma_j \pm i$	1	$1 + S(\gamma_j) + S(i)$

1.2.2 Представяне на данни в паметта

Представянето на числа в RAM модела става чрез запис в определена k -ична бройна система. Числото k е със семантиката на броя различни единици информация, които машината различава. (при машините на Тюринг k е размерът на азбуката, чиито символи пишем върху лентата).

Важна характеристика в RAM модела е т.н. размер на машинната дума: броя единици информация, които една клетка от паметта съдържа. На този етап се вижда значението на числото k . Нека размерът на машинната дума бележим с d . Тогава:

- При $k = 1$ най-голямото число, което може да се запише в една клетка, е d . Записване на числото n в паметта изисква $\lceil n/d \rceil$ клетки.
- При $k > 1$ най-голямото число, което може да се запише в една клетка, е k^d . Записване на числото n в паметта изисква $\lceil n/k^d \rceil$ клетки.

Както се вижда, разликата е експоненциална. Ние ще работим с машини, в които представянето на числата е в $k > 1$ -ична бройна система.

1.2.3 Операции за константно време в RAM модела

Следните операции върху числа могат да се изпълнят за константно време в RAM модела, при условие, че операндите се побират в една машинна дума:

- $a + b, -a, a * b, a/b$ ($b \neq 0$)
- $a^b, \lfloor \log_b a \rfloor, a!$ (в случаите, когато $a!$ се побира в машинната дума)
- $a \div b$ (целочислено), $a \bmod b$
- $a = b, a < b$ (може да считаме, че това са операции, чиито резултат е 0 или 1)
- $a \vee b, \neg a, a >> b, a << b$

Разбира се, това не са всички операции. Има доста такива, които могат да се изразят като суперпозиция на изложените (например $a \leq b = a < b \vee a = b$).

1.2.4 Псевдокод

Използването на RAM програми за описване на алгоритмите в курса ще бъде доста тромаво. Затова ще представим, чрез RAM модела някои познати програмни конструкции като if then else, for, while, псевдонимите за *променливи*. Кодът, който ще пишем и анализираме ще използва тези конструкции (и други, дефинирани, когато е уместно).

Ще пишем *псевдокод* - улеснен запис на програма в RAM модела.

- Относно псевдонимите за променливи. В известните от досегашните курсове програмни езици като C++ или Java е въведено понятието за *променлива*. В RAM модела *променливите* представляват клетките от паметта и регистрите. Вместо да ги достъпваме с $\rho(i)$ или γ_j ще използваме подходящи имена (псевдоними).
- Относно if condition then option1 else option2.
- Относно for $i = start$ to end with step do body done.
- Относно while condition do body done.

1.3 Първи пример

Ще разгледаме добре известния алгоритъм на *Евклид* за намиране на най-голям общ делител на две неотрицателни естествени числа.

Задача 1.1 (GCD).

Вход: $A, B \in \mathbb{N}$.

Изход: $\text{НОД}(A, B)$.

EUCLIDGCD(A, B : non-negative integers)

```

@1  while  $A > 0 \wedge B > 0$  do
@2    if  $A > B$  then
@3       $A \leftarrow A \bmod B$ 
@4    else
@5       $B \leftarrow B \bmod A$ 
@6    end if
@7  end while
@8  return  $\max\{A, B\}$ 

```

Твърдение 1.1. При вход естествени числа A и B , $\text{EUCLIDGCD}(A, B)$ връща техния най-голям общ делител.

Лема 1.2. Ако $d = \text{НОД}(A, B)$ и A_n и B_n са състоянията съответно на A и B при n -тото достигане на ред @1 на EUCLIDGCD , то $\text{НОД}(A_n, B_n) = d$.

Колко са операциите, които алгоритъмът извършва върху следните входове:

- $A = 0, B = 3$
- $A = 64, B = 32$
- $A = 13, B = 21$
- $A = f_n, B = f_{n+1}$ (f_n е n -тото число на Фибоначи)

EUCLIDGCDREC(A, B : non-negative integers)

```

@1  if  $A = 0 \vee B = 0$  then
@2    return  $\max\{A, B\}$ 
@3  end if
@4  if  $A > B$  then
@5    return  $\text{EuclidGCDRec}(A \bmod B, B)$ 
@6  end if
@7  return  $\text{EuclidGCDRec}(A, B \bmod A)$ 

```

Твърдение 1.3. При вход естествени числа A и B , $\text{EUCLIDGCDREC}(A, B)$ връща техния най-голям общ делител.

Лема 1.4. За произволно естествено n , при вход A, B , такива, че $A + B = n$ е изпълнено, че $\text{EUCLIDGCDREC}(A, B)$ връща най-малкото общо кратно на A и B .

Рекурсивната версия **EUCLIDGCDREC** позволява съчно намиране на числата от тъждеството на Безу:

$$\begin{aligned}
 A' &= A - Bq, & B' &= B \\
 u'A' + v'B' &= (A, B) \\
 u'(A - Bq) + v'B_n &= (A, B)
 \end{aligned}$$

$$u'A + (v' - u'q)B = (A, B)$$

$$\boxed{u = u' \quad v = v' - u'q}$$

1.4 Коректност на алгоритъм

Искаме алгоритмите, които пишем да гарантират, че върнатата стойност за съответния *екземпляр* на изчислителната задача, да е сред множеството от възможни *решения*.

За целта доказваме свойството *коректност* за алгоритмите си.

1.4.1 Итеративен подход

Основава се на *изобретяването* на инварианти: твърдения за състоянието на променливите и масивите, което остава вярно през целия ход на алгоритъма.

1.4.2 Рекурсивен подход

При доказването на *коректност* на алгоритми, които използват рекурсия се използва метода на математическата индукция по *свойство* на входа.

1.5 Времева сложност

Времева сложност на алгоритъм $\mathcal{A}$ при вход $Input$ наричаме броя елементарни операции, които $\mathcal{A}$ извършва върху $Input$, за да завърши.

Времева сложност в най-лошия случай на $\mathcal{A}$ е функция $Time_{\mathcal{A}}(n)$, приемаща големина на входа n и връщаща максималния брой операции, които $\mathcal{A}$ може да извърши върху вход с големина n .

Ще покажем, че действително входът, при който $EuclidGCD$ извършва най-много операции, е пряко обвързан с числата на Фибоначи.

Лема 1.5. Ако при вход A и B $EuclidGCD$ достига ред $\Theta 1$ точно k пъти, то $\min\{A, B\} \geq f_{k-1}$.

Твърдение 1.6. Времевата сложност на $EuclidGCD$ при вход $(A > 0, B > 0)$ е не повече от $6 \log_{\varphi}(\min\{A, B\}) + 14$.

(разликата спрямо упражнението е, че там е пропусната операцията присвояване на стойност)

1.6 Асимптотика

Имаме горна граница за времевата сложност на $EuclidGCD$ в най-лошия случай. Такъв аргумент обаче би бил тромав за следене при по-сложни алгоритми. За целта въвеждаме следната класификация на функциите $\mathbb{N}^+ \rightarrow \mathbb{R}$:

Определение 1.5 (Асимптотично сравнение - $\preceq$).

$$f \preceq g \stackrel{def}{\iff} \exists N \in \mathbb{N} \exists c \in \mathbb{R}^+ \forall n (n > N \rightarrow f(n) \leq cg(n))$$

За нас стремеж ще бъде алгоритмите, които пишем да имат "възможно най-малка" относно $\preceq$ сложност в най-лошия случай.

Определение 1.6 (*Big O*). Класът на асимптотично не по-големите функции от f е множеството

$$\mathcal{O}(f) = \{g \mid g \preceq f\}$$

Определение 1.7 (*Big Ω*). Класът на асимптотично не по-малките функции от f е множеството

$$\Omega(f) = \{g \mid f \preceq g\}$$

Определение 1.8 (*Big Θ*). Класът на асимптотично равните функции на f е множеството

$$\Theta(f) = \mathcal{O}(f) \cap \Omega(f)$$

Под записа $f = X(g)$, където $X \in \{\mathcal{O}, \Omega, \Theta\}$, ще имаме предвид $f \in X(g)$.

1.6.1 Основни свойства

1. За всяко $c \in \mathbb{R}^+$ е в сила, че $cf = \Theta(f)$
2. Ако $f = \mathcal{O}(g)$ и $g = \mathcal{O}(h)$, то $f = \mathcal{O}(h)$
3. Ако $f_1 = \mathcal{O}(g_1)$ и $f_2 = \mathcal{O}(g_2)$, то $f_1 + f_2 = \mathcal{O}(\max\{g_1, g_2\})$
4. Ако $f_1 = \mathcal{O}(g_1)$ и $f_2 = \mathcal{O}(g_2)$, то $f_1 f_2 = \mathcal{O}(g_1 g_2)$
5. Ако съществува границата $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = L$, то:
 - (а) $L < \infty$ т.с.т.к. $f \in \mathcal{O}(g)$
 - (б) $0 < L < \infty$ т.с.т.к. $f \in \Theta(g)$
 - (в) ако $L = \infty$, то $f \in \Omega(g)$ (Обратното не винаги е вярно!)
6. За произволни $a > 1$ и $k \geq 0$ е в сила, че $n^k = \mathcal{O}(a^n)$
7. За произволни $a > 1$ и $k > 0$ е в сила, че $\log_a n = \mathcal{O}(n^k)$
8. Ако $f = \Theta(g)$, то $\log_a f = \Theta(\log_a g)$

Релацията $\preceq$ е преднаредба; не всеки две функции са сравними (пример са $\sin(n)$ и $\cos(n)$).

Твърдение 1.7. Времевата сложност в най-лошия случай на `EuclidGCD` при вход (A, B) е $\mathcal{O}(\log(\min\{A, B\}))$.

2 Линејни обхождания на масив

2.1 Сложност по памет

Сума на размерите на заделените променливи и масиви по време на работа на алгоритъма $\mathcal{A}$ върху даден вход.

Заделянето на масив с размер n в псевдокод ще означаваме с:

SOMEALG(Input)

```

@1  ...
@2  A[1...n] ← malloc(n): ARRAY OF <TYPE>
@3  ...

```

Такова заделяне на памет извършва $\Theta(S(A[1...n]))$ елементарни операции.

Задача 2.1 (MAXIMUM SUBARRAY).

Търсим инфикс на масив с максимална сума. За целта разглеждаме т.н. алгоритъм на Kadane.

Вход: $A[1...n]$ - масив от рационални числа.

Изход: $\max \left\{ \sum_{k=i}^j A[k] \mid 1 \leq i \leq j \leq n \right\}$ - максимална сума на непразен инфикс на A .

KADANE($A[1...n]$: array of rationals)

```

@1  maxInfix ←  $-\infty$ 
@2  maxSuffix ← 0
@3  for i = 1 to n do
@4    maxSuffix ← maxSuffix + A[i]
@5    maxInfix ← max {maxSuffix, maxInfix}
@6    maxSuffix ← max {0, maxSuffix}
@7  end for
@8  return maxInfix

```

Твърдение 2.1. При подаден на вход масив от рационални числа $A[1...n]$, Kadane връща максималната сума на непразен последователен подмасив на A .

В доказателството на **Твърдение 2.1** ще използваме следния **Инвариант**:

Ако при k -тото достигане на ред @3 състоянията на $maxSuffix$ и $maxInfix$ са съответно $maxSuffix_k$ и $maxInfix_k$, то

$$maxInfix_k = \max \left\{ \sum_{t=i}^j A[t] \mid 1 \leq i \leq j \leq k-1 \right\} \text{ и}$$

$$maxSuffix_k = \max \left\{ \sum_{t=i}^k A[t] \mid 1 \leq i \leq k \right\}$$

Твърдение 2.2. Времевата сложност на KADANE в най-лошия случай е $\Theta(n)$.

Друго възможно линейно решение използва т.н. *префиксни суми* и се опира на следните наблюдения:

1. Всяка инфиксна сума е разлика на 2 префиксни:

$$\sum_{k=i}^j A[k] = \sum_{k=1}^j A[k] - \sum_{k=1}^{i-1} A[k]$$

2. Тогава най-голямата инфиксна сума се намира лесно чрез:

$$\max_{1 \leq i \leq j \leq n} \left\{ \sum_{t=i}^j A[t] \right\} = \max_{1 \leq i \leq j \leq n} \left\{ \sum_{t=1}^j A[t] - \sum_{t=1}^{i-1} A[t] \right\} = \max_{1 \leq j \leq n} \left\{ \sum_{t=1}^j A[t] - \min_{1 \leq i \leq j} \sum_{t=1}^{i-1} A[t] \right\}$$

Задача 2.2 (DISTANT PAIR).

Дадени са n точки в равнината. Да се намери най-голямото манхатънско разстояние между 2 точки.

Точка ще представяме чрез записа:

```
Point = {
    x : rational
    y : rational
}
```

Манхатънското разстояние между записи от тип *Point* $P1$ и $P2$ ще бележим с $d_M(P1, P2) = |P1.x - P2.x| + |P1.y - P2.y|$

Вход: $P[1 \dots n]$ - масив от *Points*.

Изход: $\max \{d_M(P[i], P[j]) \mid 1 \leq i, j \leq n\}$.

DISTANT($P[1 \dots n]$: array of Points)

```

01  maxSum ← -∞
02  minSum ← +∞
03  maxDiff ← -∞
04  minDiff ← +∞
05  for i = 1 to n do
06    maxSum ← max {P[i].x + P[i].y, maxSum}
07    minSum ← min {P[i].x + P[i].y, minSum}
08    maxDiff ← max {P[i].x - P[i].y, maxDiff}
09    minDiff ← min {P[i].x - P[i].y, minDiff}
10  end for
11  return max {maxSum - minSum, maxDiff - minDiff}
```

Лема 2.3.

$$\max_{1 \leq i, j \leq n} \{d_M(P[i], P[j])\} = \max \left\{ \max_{1 \leq i, j \leq n} \{(P[i].x + P[i].y) - (P[j].x + P[j].y)\}, \right. \\ \left. \max_{1 \leq i, j \leq n} \{(P[i].x - P[i].y) - (P[j].x - P[j].y)\} \right\}$$

Твърдение 2.4. При вход масив $P[1 \dots n]$, масив от записи *Point*, DISTANT връща най-голямото манхатънско разстояние между някои две от точките за време $\Theta(n)$.

Задача 2.3 (SIEVE). Да се намерят простите числа $\leq n$ (решето на Ератостен)

Вход: n - положително естествено число

Изход: $P[1...k] = \{p \mid p \leq n \text{ \& } p \text{ - просто}\}$.

Идея: Ще използваме, че

$$p \text{ е просто т.с.т.к. } \forall k < p \forall q < p (q \text{ е просто} \rightarrow kq \neq p)$$

SIEVE(n : positive integer)

01 $A[1...n] \leftarrow \text{malloc}(n)$: ARRAY OF BOOLEANS

02 for $i = 2$ to n do

03 $A[i] \leftarrow \text{true}$

04 end for

05 $A[1] \leftarrow \text{false}$

06 for $i = 2$ to n do

07 if $A[i] = \text{true}$ then

08 $\text{Eliminate}(i, A[2...n])$

09 end if

010 end for

011 $P[1...k] \leftarrow \text{ArgTrue}(A[1...n])$

012 return $P[1...k]$

ELIMINATE(p : prime, $A[1...n]$: booleans)

01 $j \leftarrow 2 * p$

02 while $j \leq n$ do

03 $A[j] \leftarrow \text{false}$

04 $j \leftarrow j + p$

05 end while

COUNTTRUE($A[1...n]$: array of booleans)

01 $k \leftarrow 0$

02 for $i = 1$ to n do

03 if $A[i] = \text{true}$ then

04 $k \leftarrow k + 1$

05 end if

06 end for

07 return k

ARGTRUE($A[1...n]$: array of booleans)

01 $k \leftarrow \text{CountTrue}(A[2...n])$

02 $P[1...k] \leftarrow \text{malloc}(k)$: ARRAY OF INTEGERS

03 $k \leftarrow 1$

04 for $i = 2$ to n do

05 if $A[i] = \text{true}$ then

06 $P[k] \leftarrow i$

07 $k \leftarrow k + 1$

08 end if

09 end for

010 return $P[1...k]$

Твърдение 2.5. При вход просто число p и масив от булеви стойности $A[1...n]$, ELIMINATE модифицира $A[1...n]$ до $A'[1...n]$, където за $1 \leq i \leq n$:

$$A'[i] = \begin{cases} \text{false} & , \text{ ако } p|i \\ A[i] & , \text{ иначе} \end{cases}$$

Още нещо, ELIMINATE завършва за време $\Theta\left(\frac{n}{p}\right)$.

Твърдение 2.6. При вход положително цяло число n , SIEVE връща масив, съдържащ простите числа $p \leq n$. Още нещо, SIEVE завършва за $\mathcal{O}(n \log n)$ време.

За доказателството на **Твърдение 2.6**:

1. Инвариант: При всяко достигане на 07 на SIEVE, за всяко $1 \leq l \leq i$ е в сила, че $A[l] = \text{true}$ т.с.т.к. l е просто.

Вярността на тази инварианта съществено ползва **Твърдение 2.5**, както и свойството, че след извикване ELIMINATE($p, A[1...n]$), за масива $A'[1...n]$ е изпълнено, че

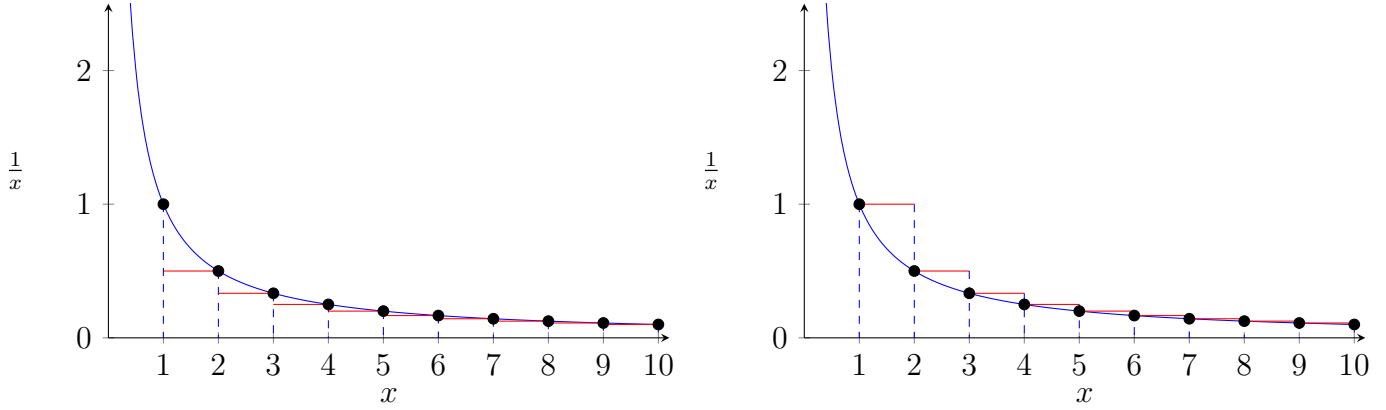
$$A[i] = \bigwedge_{\substack{q \leq p \\ q \text{ - просто}}} q \nmid i$$

2. COUNTTRUE и ARGTRUE работят за време $\Theta(n)$ и връщат съответно броя и масив от простите числа $p \leq n$

3. SIEVE работи в най-лошия случай за време

$$\mathcal{O}(n) + \sum_{i=2, i \text{ - просто}}^n \text{Time}_{\text{Eliminate}}(p, A[1\dots n]) + \Theta(n) + \Theta(n) = \dots = \mathcal{O}(n \log n)$$

Идея за доказване на $\sum_{i=2}^n \frac{1}{i} = \Theta(\log n)$:



Твърдение 2.7 (техника за работа със сума от монотонна непрекъсната функция). Нека $f : \mathbb{N}^+ \rightarrow \mathbb{R}^+$ монотонна непрекъсната функция и нека

$$S = \sum_{i=1}^n f(i)$$

$$I = \int_1^n f(x) dx$$

Тогава:

- ако f е растяща, то $I + f(1) \leq S \leq I + f(n)$.
- ако f е намаляваща, то $I + f(n) \leq S \leq I + f(1)$.

($n e^{n^2}$ е пример за неподходяща употреба)

Допълнение 2.1 ($(\mathcal{O}(n \log(\log n)))$ е по-добра оценка за SIEVE)).

Използваме наготово, че $\pi(k) = |\{p \mid p \leq k \text{ \& } p \text{ е просто}\}| = \frac{k}{\log(k)} \left(1 + \mathcal{O}\left(\frac{1}{\log(k)}\right)\right)$,

$$\begin{aligned} \sum_{p \leq n} \frac{1}{p} &= \sum_{k=1}^n \frac{\pi(k) - \pi(k-1)}{k} = \sum_{k=1}^n \frac{\pi(k)}{k} - \sum_{k=0}^{n-1} \frac{\pi(k)}{k+1} \\ &= \frac{\pi(n)}{n} + \sum_{k=1}^{n-1} \frac{\pi(k)}{k(k+1)} = \mathcal{O}(1) + \sum_{k=1}^{n-1} \frac{\pi(k)}{k^2} - \sum_{k=1}^{n-1} \frac{\pi(k)}{k^2(k+1)} \\ &= \sum_{k=3}^{n-1} \frac{\pi(k)}{k^2} + \mathcal{O}(1) = \sum_{k=3}^{n-1} \left[\frac{1}{k \log(k)} + \mathcal{O}\left(\frac{1}{k \log(k)^2}\right) \right] + \mathcal{O}(1) \\ &= \log(\log(n)) + \mathcal{O}(1) \end{aligned}$$

Инициализацията на @7 задава ненужно малка стойност за j . Оптимално е тя да бъде $i * i$, защото съставните числа по-малки от $i * i$ имат прост делител $< i$, тоест вече са отпаднали

като кандидати за прости числа ($A[k] = false$). Това е мотивация за "подобрен" алгоритъм (асимптотично нещата не се променят):

ELIMINATE2($p : prime, A[1..n] : booleans$)

```

01   $j \leftarrow p * p$ 
02  while  $j \leq n$  do
03     $A[j] \leftarrow false$ 
04     $j \leftarrow j + p$ 
05  end while

```

Задача 2.4 (Линейно решето). Съществува решение със сложност $O(n)$. Макар по-добрата си времева сложност, то често не се използва на практика заради допълнителната памет, която е необходима.

Идея: за всяко число в интервала $[2, n]$ да се намери най-малкия му прост делител в масива lp . Нека с $lp(x)$ означим най-малкия прост делител на числото x . Тогава всяко съставно число по единствен начин може да се представи като произведение на $lp(x) * t$, където $t > 1$ и $lp(t) \geq lp(x)$.

Така подобно на предишното решение на задачата, итерирайки по числата от 2 до n , можем за всяко да проверим дали сме намерили $lp[i]$ вече. Ако не сме ($lp[i] = 0$), то значи i е просто и можем да запишем $lp[i] = i$. След това, **независимо дали i е просто** записваме $lp[x] = lp(x)$ за всяко x , което се представя във вида $x = lp(x) * i$.

Ето и псевдокод, представящ горната идея: Да се намерят простите числа $\leq n$ (линейно решето на Ератостен)

Вход: n - положително естествено число

Изход: $P[1..k] = \{p \mid p \leq n \text{ \& } p \text{ - просто}\}$.

LINEARSIEVE($n : positive\ integer$)

```

01   $lp[1..n] \leftarrow malloc(n): \text{ARRAY OF INTEGERS}$ 
02   $pr[1..n] \leftarrow malloc(n): \text{ARRAY OF INTEGERS}$ 
03   $k \leftarrow 0$ 
04  for  $i = 2$  to  $n$  do
05     $lp[i] \leftarrow 0$ 
06  end for
07  for  $i = 2$  to  $n$  do
08    if  $lp[i] = 0$  then
09       $k \leftarrow k + 1$ 
10      $pr[k] \leftarrow i$ 
11      $lp[i] \leftarrow i$ 
12    end if
13    MarkLP( $i, lp[1..n], pr[1..k]$ )
14  end for
15  return  $pr[1..k]$ 

```

MARKLP($t : integer, lp[1..n] : integers, pr[1..k] : array$)

```

01   $j \leftarrow 1$ 
02  while  $pr[j] * t \leq n \wedge pr[j] \leq lp[t]$  do
03     $lp[pr[j] * t] \leftarrow pr[j]$ 
04     $j \leftarrow j + 1$ 
05  end while

```

Твърдение 2.8. При вход естествено число $t > 1$ и масиви $lp[1..n]$ и $pr[1..k]$, съдържащи простите числа от 1 до t , MarkLP модифицира масива lp до lp' , така че:

$$lp'[i] = \begin{cases} lp(i) & , \text{ ако } i/lp(i) = t \\ lp[i] & , \text{ иначе} \end{cases}$$

Твърдение 2.9. При вход положително цяло число n , **LINEARSIEVE** връща масив, съдържащ простите числа $p \leq n$. Още нещо, **LINEARSIEVE** завършва за $\mathcal{O}(n)$ време.

За доказателството на **Твърдение 2.9**:

1. Инвариант: При всяко достигане на **@7** на **LINEARSIEVE**, за всяко $1 \leq l \leq i$ е в сила, че $\overline{lp[l]} = lp(i)$.
Вярността на тази инварианта съществено ползва **Твърдение 2.8**.
2. Стойността на $lp[i]$ се променя точно веднъж за всяко i при изпълнението на **LINEARSIEVE**.
3. **LINEARSIEVE** работи в най-лошия случай за време $\mathcal{O}(n)$.

3 Сортиране

Цел: подредба на елементите "по големина" (т.е. по някакво тяхно свойство)

Алгоритми за сортиране:

- $\mathcal{O}(n^2)$ в най-лошия случай
 - SELECTIONSORT
 - INSERTIONSORT
 - QUICKSORT (при случаен *pivot*, средна сложност $\mathcal{O}(n \log n)$)
- $\mathcal{O}(n \log n)$
 - HEAPSORT
 - MERGESORT
 - QUICKSORT (при *pivot*, определен като "Median of medians")
- $\mathcal{O}(n + d) \rightarrow$ COUNTINGSORT (използваме, когато имаме горна граница d за елементите на масива)

Задача 3.1 (2-SUM).

Вход: $A[1\dots n]$ - масив от рационални числа.

Изход: Има ли $1 \leq i < j \leq n$, такива, че $A[i] + A[j] = 0$?

Ще разгледаме първо опростената версия на задачата, когато входът е сортиран:

TWOPOINTERSUM($A[l\dots r]$: sorted array of rationals, $target$: rational number)

```

@1   $i \leftarrow l$ 
@2   $j \leftarrow r$ 
@3  while  $i < j \wedge A[i] + A[j] \neq target$  do
@4    if  $A[i] + A[j] < target$  then
@5       $i \leftarrow i + 1$ 
@6    else
@7       $j \leftarrow j - 1$ 
@8    end if
@9  end while
@10 return  $i < j$ 
```

Твърдение 3.1. При вход сортиран масив $A[1\dots n]$ и рационално число $target$, TWOPOINTERSUM връща истина т.с.т.к. има $1 \leq i < j \leq n$, такива, че $A[i] + A[j] = target$.

Още нещо, TWOPOINTERSUM работи върху всеки свой вход за време $\Theta(n)$.

2SUM($A[1\dots n]$: array of rationals)

```

@1  MERGESORT( $A[1\dots n]$ )
@2  return TWOPOINTERSUM( $A[1\dots n]$ , 0)
```

Коректността на 2SUM е директно следствие от **Твърдение 3.1**, тъй като подаденият на @2 масив е сортираният вход и е в сила, че такива индекси $1 \leq i, j \leq n$ има в A т.с.т.к. има такива индекси $1 \leq i', j' \leq n$ в сортираната пермутация A' на A .

Задача 3.2 (3-SUM).

Вход: $A[1\dots n]$ - масив от рационални числа.

Изход: Има ли $1 \leq i < j < k \leq n$, такива, че $A[i] + A[j] + A[k] = 0$?

3SUM($A[1\dots n]$: array of rationals)

```

01  MERGESORT( $A[1\dots n]$ )
02   $i \leftarrow 1$ 
03  while  $i < n - 1 \wedge \neg \text{TwoPointerSum}(A[i+1\dots n], -A[i])$  do
04     $i \leftarrow i + 1$ 
05  end while
06  return  $i < n - 1$ 

```

Коректност на предложения алгоритъм може да се покаже например чрез записване на конюнкцията, която n -те проверки на цикъла представя.

Отворен е въпросът дали има $\mathcal{O}(n^{2-\epsilon})$ алгоритъм, решаващ **3-SUM**.

Задача 3.3 (4-SUM).

Вход: $A[1\dots n]$ - масив от рационални числа.

Изход: Има ли $1 \leq i < j < k < l \leq n$, такива, че $A[i] + A[j] + A[k] + A[l] = 0$?

Ясно е как произползвайки 3SUM можем да решим **4-SUM** за време $\mathcal{O}(n^3)$. Ако обаче отпадне условието индексите да са различни, може да постъпим по-хитро, произползвайки само 2SUM.

4SUM($A[1\dots n]$: array of rationals)

```

01   $B[1\dots 2n^2] \leftarrow \text{malloc}(2n^2)$ : ARRAY OF rationals
02   $k \leftarrow 1$ 
03  for  $i = 1$  to  $n$  do
04    for  $j = 1$  to  $n$  do
05       $B[k] \leftarrow A[i] + A[j]$ 
06       $B[k+1] \leftarrow A[i] + A[j]$ 
07       $k \leftarrow k + 2$ 
08    end for
09  end for
10  MERGESORT( $B[1\dots 2n^2]$ )
11  TwoPointerSum( $B[1\dots 2n^2]$ , 0)

```

4SUM е коректен, защото в $A[1\dots n]$ има четворка индекси със сума на съответните елементи 0 т.с.т.к. в $B[1\dots 2n^2]$ има двойка индекси със сума на съответните елементи 0.

Времева сложност на 4SUM: $\mathcal{O}(2n^2) + \mathcal{O}(2n^2) + \mathcal{O}(2n^2 \log(2n^2)) + \mathcal{O}(2n^2) = \mathcal{O}(n^2 \log n)$.

Задача 3.4 (MINIMUM ABSOLUTE SUBARRAY). *Да се намери инфиксна сума в масив с най-малка абсолютна стойност.*

(Стока в магазин, да няма недостиг, но и да няма излишък; в кой период е постигнат оптимален резултат?)

Вход: $A[1\dots n]$ - масив от рационални числа.

Изход: $\min \left\{ \left| \sum_{k=i}^j A[k] \right| \mid 1 \leq i \leq j \leq n \right\}$ - най-близка до 0 сума на непразен инфикс на A .

Лема 3.2. Ако за $k < l$ имаме, че

$$|S_k - S_l| = \min \{|S_j - S_i| \mid 0 \leq i < j \leq n\}$$

то няма q , такова, че $S_k < S_q < S_l$ или $S_l < S_q < S_k$.

Сега след **Лема 3.2** остава да съобразим, че е достатъчно да сортираме префиксните суми и да търсим минимална абсолютна разлика между някои два последователни.

MINABSSUBARRAY($A[1..n]$: array of rationals)

```

@1  B[0...n] ← malloc(n + 1): ARRAY OF rationals
@2  B[0] ← 0
@3  for i = 1 to n do
@4    B[i] ← B[i - 1] + A[i]
@5  end for
@6  MERGESORT(B[0...n])
@7  m ← +∞
@8  for i = 1 to n do
@9    m ← min {m, B[i] - B[i - 1]}
@10 end for
@11 return m

```

Твърдение 3.3. При вход $A[1..n]$ - масив от рационални числа, MINABSSUBARRAY работи за време $\mathcal{O}(n \log n)$ и връща най-малката абсолютна стойност на сума на елементите на непразен подмасив на A .

Задача 3.5 (TRIANGLES).

Вход: $A[1..n]$ - масив от рационални числа.

Изход: $|\{ \langle i, j, k \rangle \mid 1 \leq i < j < k \leq n \ \& \ A[i], A[j], A[k] \text{ са страни на триъгълник} \}|$

TRIANGLES($A[1..n]$: array of rationals)

```

@1  MERGESORT(A[1...n])
@2  count ← 0
@3  for i = 1 to n - 2 do
@4    k ← i + 2
@5    for j = i + 1 to n - 1 do
@6      while k < n ∧ A[i] + A[j] > A[k] do
@7        k ← k + 1
@8      end while
@9      count ← count + (k - j - 1)
@10 end for
@11 end for
@12 return count

```

Коректност на TRIANGLES може да се покаже с инварианта за цикъла на @3, като за нейното доказване е удобно в стъпката да се дефинира друга инварианта за цикъла на @5.

Относно времевата сложност на **TRIANGLES**, ключово наблюдение е **Q9** се изпълнява най-много $n - i - 2$ пъти за всяко изпълнение на тялото на цикъла на ред **Q3**...

$$Time_{Triangles} = \sum_{i=1}^{n-2} (\mathcal{O}(n - i - 2) + \mathcal{O}(n - i - 2)) = \dots = \mathcal{O}(n^2)$$

Задача 3.6 (GROUPING).

Разглеждаме редица от $2n$ рационални числа $\{a_i\}_{i=1}^{2n}$. Групиране на $\{a_i\}_{i=1}^{2n}$ ще наричаме множество от наредени двойки (a_p, a_q) , такова, че всеки член a_i на редицата участва точно в една наредена двойка и то веднъж. Тежест на групиране P ще наричаме

$$w(P) = \max \{a_p + a_q \mid (a_p, a_q) \in P\}$$

. Целим да намерим минималната възможна тежест измежду всички групирания.

Вход: $A[1\dots 2n]$ - представяне на редица от рационални числа.

Изход: $m = \min \{w(P) \mid P \text{ е групиране за } A[1\dots 2n]\}$.

Лема 3.4. Минимална тежест на растяща редица $\{a_i\}_{1 \leq i \leq n}$ се достига за групирането

$$\{(a_i, a_{2n-i+1}) \mid 1 \leq i \leq n\}$$

Доказателство. Допускаме, че $\{(a_i, a_{2n-i+1}) \mid 1 \leq i \leq n\}$ не е оптимално групиране и нека P е такова с тежест $a_x + a_y$, т.е.

$$a_x + a_y < a_{i_0} + a_{2n-i_0+1}$$

Нека $q \leq i_0$ и $(a_p, a_{2n-q+1}) \in P$.

Тогава $a_p + a_{2n-q+1} < a_x + a_y < a_{i_0} + a_{2n-i_0+1}$, но $a_{2n-q+1} \geq a_{2n-i_0+1}$, следователно $a_p < a_{i_0}$.

Сега в групирането P всяко $1 \leq q \leq i_0$, $2n - q + 1$ има за партньор някое $1 \leq p < i_0$, което е невъзможно. $\square$

Очевидно пермутирането на входа не променя всички възможни групирания, а следователно и това с най-малка тежест не се променя.

След придобитата интуиция за отговора следва само да се възползваме от сортирането:

LIGHTESTGROUP($A[1\dots 2n]$: array of rationals)

```

Q1  HEAPSORT(A[1...2n])
Q2  maxSum ← -∞
Q3  for i = 1 to n do
Q4    maxSum ← max {maxSum, A[i] + A[2n - i + 1]}
Q5  end for
Q6  return maxSum

```

Времева сложност: $\mathcal{O}(n \log n)$.

Оказва се, че това групиране решава и следните подобни проблеми:

1. Лекота на групиране P ще наричаме

$$l(P) = \min \{a_p + a_q \mid (a_p, a_q) \in P\}$$

Вход: $A[1\dots 2n]$ - представяне на редица от рационални числа.

Изход: $m = \max \{l(P) \mid P \text{ е групиране за } A[1\dots 2n]\}$.

2. Потенциал на групиране P ще наричаме

$$\pi(P) = w(P) - l(P)$$

Вход: $A[1..2n]$ - представяне на редица от рационални числа.

Изход: $m = \min \{ \pi(P) \mid P \text{ е групиране за } A[1..2n] \}$.

Задача 3.7 (Задача 1, писмен изпит, редовна сесия, 2023г.).

Представяне на редица от n интервала $I_1, \dots, I_n$ ще наричаме двойка от масиви $L[1..n]$ и $R[1..n]$, за които:

$$I_i = [L[i], R[i]] \text{ за всяко } i \in \{1, 2, \dots, n\}$$

1. Разглеждаме следния проблем **INTERSECTINGPAIRS**:

Вход: $L[1..n]$, $R[1..n]$ представяне на редица от n интервала $I_1, I_2, \dots, I_n$

Изход: $N = | \{ (i, j) \mid i < j \text{ и } I_i \cap I_j \neq \emptyset \} |$

Да се предложи алгоритъм със сложност $\mathcal{O}(n \log n)$, който решава проблема **INTERSECTINGPAIRS**.

Да се докаже коректността и времевата сложност на предложения алгоритъм.

2. За редица от интервали $I = (I_1, I_2, \dots, I_n)$ с $m(I)$ означаваме най-големия брой интервали от редицата, които в съвкупност имат непразно сечение.

Разглеждаме следния проблем **MAXINTERSECTION**:

Вход: $L[1..n]$, $R[1..n]$ представяне на редица от n интервала $I_1, I_2, \dots, I_n$

Изход: $m(I)$

Да се предложи алгоритъм със сложност $\mathcal{O}(n \log n)$, който решава проблема **MAXINTERSECTION**.

Да се докаже коректността и времевата сложност на предложения алгоритъм.

Удобно е използването на запис

```
Endpoint = {
  x : rational
  closing : boolean
}
```

Забелязваме, че сортиране на тези записи лексикографски (*false* считаме за "по-малко" от *true*, за да "засечем" едноточкови пресичания) би помогнало за решаване и на двата проблема за линейно време след това. За целта нека **MERGESORTENDPOINTS** е модификация на **MERGESORT**, получаваща на вход масив от **Endpoints** и връщаща негова сортирана пермутация по контекст $e1 \preceq e2$ т.с.т.к. $e1.x < e2.x \vee (e1.x = e2.x \wedge e1.closing = false)$.

1. INTERSECTINGPAIRS:

INTERSECTINGPAIRS($L[1..n]$: left ends, $R[1..n]$: right ends)

```

01   $A[1..2 * n] \leftarrow \text{malloc}(2 * n)$ : ARRAY of Endpoints
02  for  $i = 1$  to  $n$  do
03     $A[2 * i - 1] \leftarrow \text{Endpoint}(x \leftarrow L[i], \text{closing} \leftarrow false)$ 
04     $A[2 * i] \leftarrow \text{Endpoint}(x \leftarrow R[i], \text{closing} \leftarrow true)$ 
05  end for
06  MERGESORTENDPOINTS( $A[1..2n]$ )
07   $active \leftarrow 0$ 
```

```

@8  intersections ← 0
@9  for i = 1 to 2 * n do
@10  if A[i].closing = true then
@11    active ← active - 1
@12  else
@13    intersections ← intersections + active
@14    active ← active + 1
@15  end if
@16 end for
@17 return  intersections

```

Твърдение 3.5. При вход представяне на интервали $I_1, \dots, I_n$ с $L[1..n], R[1..n]$, INTERSECTINGPAIRS връща броя на двойките различни интервали, които се пресичат.

Аргументация:

Нека полетата $\mathbf{x}$ в сортирания на @6 $A'[1..2n]$ са

$$\underbrace{x_{1,1}, x_{1,2}, \dots, x_{1,c_1}}_{c_1}, \underbrace{x_{2,1}, x_{2,2}, \dots, x_{2,c_2}}_{c_2}, \dots, \underbrace{x_{t,1}, x_{t,2}, \dots, x_{t,c_t}}_{c_t}$$

където $x_{i,p} = x_{j,q}$ т.с.т.к. $i = j$.

Тогавя, ако $active_k$ и $intersections_k$ са състоянията на $active$ и $intersections$ непосредствено преди обработването на $x_{k,1}$ (при $1 + c_1 + c_2 + \dots + c_{k-1}$ -вото достигане на @9), то

$$active_k = |\{I_i \mid L[i] < x_{k,1} \ \& \ R[i] \geq x_{k,1}\}|$$

$$intersections_k = |\{(i, j) \mid L[i] < x_{k,1} \ \& \ L[j] < x_{k,1} \ \& \ I_i \cap I_j \neq \emptyset\}|$$

Относно времевата сложност:

$$Time_{IntersectingPairs} = O(2n) + O(n) + O(2n \log(2n)) + O(2n) = O(n \log n)$$

2. MAXINTERSECTION:

MAXINTERSECTION($L[1..n]$: left ends, $R[1..n]$: right ends)

```

@1  A[1...2 * n] ← malloc(2 * n): ARRAY of Endpoints
@2  for i = 1 to n do
@3    A[2 * i - 1] ← Endpoint(x ← L[i], closing ← false)
@4    A[2 * i] ← Endpoint(x ← R[i], closing ← true)
@5  end for
@6  MERGESORTENDPOINTS(A[1...2n])
@7  active ← 0
@8  maxIntersecting ← -∞
@9  for i = 1 to 2 * n do
@10  if A[i].closing = true then
@11    active ← active - 1
@12  else
@13    active ← active + 1
@14    maxIntersecting ← max {maxIntersecting, active}
@15  end if
@16 end for
@17 return  maxIntersecting

```

Твърдение 3.6. При вход представяне на интервали $I_1, \dots, I_n$ с $L[1..n], R[1..n]$, **MAXINTERSECTION** връща максималния брой интервали с непразно сечение в съвкупност.

Аргументация:

Нека полетата x в сортирания на @6 $A'[1..2n]$ са

$$\underbrace{x_{1,1}, x_{1,2}, \dots, x_{1,c_1}}_{c_1}, \underbrace{x_{2,1}, x_{2,2}, \dots, x_{2,c_2}}_{c_2}, \dots, \underbrace{x_{t,1}, x_{t,2}, \dots, x_{t,c_t}}_{c_t}$$

където $x_{i,p} = x_{j,q}$ т.с.т.к. $i = j$.

Тогава, ако $active_k$ и $intersections_k$ са състоянията на *active* и *intersections* непосредствено преди обработването на $x_{k,1}$ (при $1 + c_1 + c_2 + \dots + c_{k-1}$ -вото достигане на @9), то

$$active_k = |\{I_i \mid L[i] < x_{k,1} \ \& \ R[i] \geq x_{k,1}\}|$$

$$maxIntersecting_k = m(\{I_i \mid L[i] < x_{k,1}\})$$

Относно времевата сложност:

$$Time_{MaxIntersection} = O(2n) + O(n) + O(2n \log(2n)) + O(2n) = O(n \log n)$$

Изложеното по-долу по-скоро цели задоволяване на любопитството, отколкото необходимо знание (все пак препоръчително четиво...)

Определение 3.1 (Редукция на изчислителни задачи).

Казваме, че **PR1** се свежда до **PR2** и пишем

$$\mathbf{PR1} \lll_{f(n)} \mathbf{PR2}$$

ако има алгоритъм, разрешаващ **PR1**, който константен брой пъти използва алгоритъм, решаващ **PR2**, както и извършва $O(f(n))$ допълнителна работа.

Задача 3.8 (3-SUM-TARGET).

Вход: $A[1..n], target$ - масив от рационални числа и търсена сума.

Изход: Има ли $1 \leq i < j < k \leq n$, такива, че $A[i] + A[j] + A[k] = target$?

Твърдение 3.7 (връзка между проблемите **3-SUM** и **3-SUM-TARGET**).

Проблемите **3-SUM** и **3-SUM-TARGET** лесно се свеждат един към друг със следните алгоритми:

• **3-SUM-TARGET** $\lll_n$ **3-SUM**

- За време $O(n)$ заменяме $A[i] \rightsquigarrow A[i] - target/3$.
- Връщаме резултата от извикването на **3-SUM** при вход модифицирания масив A .

• **3-SUM** $\lll_1$ **3-SUM-TARGET**

- Връщаме резултата от извикването на **3-SUM-TARGET** при вход масива A и целева сума 0.

Задача 3.9 (3-COLLINEAR).

Вход: $P_1, \dots, P_n$ - представяния на точки в равнината.

Изход: Има ли $1 \leq i < j < k \leq n$, такива, че P_i, P_j, P_k са колинеарни?

Твърдение 3.8.

$$\mathbf{3-SUM} \lll_{n \log n} \mathbf{3-COLLINEAR}$$

Алгоритъм за **3-SUM**:

1. За време $\mathcal{O}(n \log n)$ може да се провери дали има тройка индекси, за които някои два елемента са равни и сумата на трите е 0 (как?)
2. Премахваме повтарящите се елементи
3. За всеки елемент на A конструираме $A[i] \rightsquigarrow (A[i], A[i]^3)$
4. Връщаме резултата от **3-COLLINEAR** при вход съпоставените точки.

Използваме фактът, че $A[i] + A[j] + A[k] = 0$ т.с.т.к. $\langle A[i], A[i]^3 \rangle, \langle A[j], A[j]^3 \rangle, \langle A[k], A[k]^3 \rangle$ са колинеарни.

(Второто е еквивалентно на нулева детерминанта $\begin{vmatrix} A[i] & A[i]^3 & 1 \\ A[j] & A[j]^3 & 1 \\ A[k] & A[k]^3 & 1 \end{vmatrix}$)

Задача 3.10 (CONCURRENT LINES).

Вход: $l_1, \dots, l_n$ - представяния на прави в равнината.

Изход: Има ли $1 \leq i < j < k \leq n$, такива, че l_i, l_j, l_k се пресичат в една точка?

Допълнение 3.1 (Редукция към **CONCURRENT LINES**).

$$\mathbf{3-COLLINEAR} \lll_{n \log n} \mathbf{CONCURRENT LINES}$$

Алгоритъм за **3-COLLINEAR**:

1. Сортираме по x точките за $\mathcal{O}(n \log n)$
2. Проверяваме дали има някои с еднаква x координата за $\mathcal{O}(n)$
3. Заменяме всяка точка $(x_i, y_i) \rightsquigarrow l_i : x_i x - y - y_i = 0$
4. Три точки с различни x координати са колинеарни т.с.т.к. съпоставените им прави се пресичат в 1 точка, т.е. връщаме резултата от **CONCURRENT LINES** при вход съпоставените прави.

Използваме, че $(x_i, y_i), (x_j, y_j)$ с $x_i \neq x_j$ лежат на права $l : ax - y - b = 0$ т.с.т.к. (a, b) е пресечната точка на l_i и l_j .

4 Разделяй и владей

Задача 4.1 (FLIP). $A[1...n]$, за който $A[1] > A[n]$; да се намери позицията на елемент $A[i] > A[i+1]$.

Вход: $A[1...n]$ - масив от рационални числа, за който $A[1] > A[n]$.

Изход: $i \in \mathbb{N}$, такова, че $1 \leq i < n$ и $A[i] > A[i+1]$.

Лема 4.1. Ако за редица от рационални числа $\{a_i\}_{i=1}^n$ е изпълнено, че $a_1 > a_n$, то има $1 \leq i < n$, такова, че $a_i > a_{i+1}$.

Доказателство. Допускаме противното и с индукция показваме, че за всяко $1 \leq k \leq n$ е изпълнено, че $a_1 \leq a_k$, откъдето за $k = n$ противоречието следва. $\square$

Тогава такъв индекс в масива, подаден на вход има. Удобно е да го потърсим двоично:

FLIP($A[1...n]$: array of rationals, such that $A[1] > A[n]$)

```

01  left ← 1
02  right ← n
03  while right - left > 1 do
04    m ← ⌊(left + right)/2⌋
05    if A[left] > A[m] then
06      right ← m
07    else
08      left ← m
09    end if
10  end while
11  return left

```

Твърдение 4.2. При вход $A[1...n]$, за който $A[1] > A[n]$, FLIP връща индекс i , такъв, че $1 \leq i < n$ и $A[i] > A[i+1]$. Също така FLIP има времева сложност $\mathcal{O}(\log n)$.

По-удобно за анализ е следното рекурсивно решение на същия проблем:

FLIPREC($A[l...r]$: array of rationals, such that $A[l] > A[r]$)

```

01  if r - l ≤ 1 then return l
02  m ← ⌊(l + r)/2⌋
03  if A[l] > A[m] then return FLIPREC(A[l...m])
04  return FLIPREC(A[m...r])

```

FLIPREC поражда следното уравнение за времевата си сложност:

$$T(2) \leq 2$$

$$T(n) \leq T\left(\frac{n}{2}\right) + \mathcal{O}(1)$$

...чието решение е $T(n) = \log(n)$.

Задача 4.2 (Домашно 1, писмен изпит, редовна сесия, 2023г.).

За матрица $A = (a_{i,j})_{i,j=1}^{n,n}$ от нули и единици ще казваме, че е 4-интересна, ако сумата на четирите ѝ ъглови елемента не се дели на 4, т.е., ако:

$$a_{1,1} + a_{1,n} + a_{n,1} + a_{n,n} \not\equiv 0 \pmod{4}$$

1. Да се докаже, че ако A е 4-интересна, то A съдържа подматрица 2×2 , определена от два съседни реда и два съседни стълба, която е 4-интересна.
2. Да се предложи алгоритъм с времева сложност $O(\log n)$, който решава следния проблем:

Вход: $A[1..n][1..n]$ - 4-интересна матрица

Изход: (i, j) , за които $A[i..i+1][j..j+1]$ е 4-интересна подматрица.

Използваме подобна идея:

Лема 4.3. Ако матрицата A е 4-интересна, то

$$A_{11} > A_{1n} \text{ или } A_{1n} > A_{nn} \text{ или } A_{nn} > A_{n1} \text{ или } A_{n1} > A_{11}$$

Доказателство. Директно противоречие със 4-интересността след допускане на обратното. $\square$

Ще използваме FLIP, заедно със следните негови модификации:

- FLIPCOL - получава на вход матрица $A[1..n][1..n]$, както и индекс на неин стълб j , за който $A[1][j] > A[n][j]$ и връща индекс на ред i , такъв, че $A[i][j] > A[i+1][j]$. Единствено заменяме проверката на @5 с $A[left][j] > A[m][j]$
- FLIPINV - получава на вход масив $A[1..n]$, за който $A[n] > A[1]$ и връща индекс $1 < i \leq n$, такъв, че $A[i] > A[i-1]$. Единствено заменяме проверката на @5 с $A[left] < A[mid]$.
- FLIPCOLINV - получава на вход матрица $A[1..n][1..n]$, както и индекс на неин стълб j , за който $A[1][j] < A[n][j]$ и връща индекс на ред i , такъв, че $A[i][j] > A[i-1][j]$. Единствено заменяме проверката на @5 с $A[left][j] < A[m][j]$

4INTERESTING($A[1..n][1..n]$: 0/1 matrix)

```

@1  if  $A[1][1] > A[1][n]$  then
@2     $pos \leftarrow \text{FLIP}(A[1][1..n])$ 
@3    return  $A[1..2][pos..pos+1]$ 
@4  else if  $A[1][n] > A[n][n]$  then
@5     $pos \leftarrow \text{FLIPCOL}(A[1..n][1..n], n)$ 
@6    return  $A[pos..pos+1][n-1..n]$ 
@7  else if  $A[n][n] > A[n][1]$  then
@8     $pos \leftarrow \text{FLIPINV}(A[n][1..n])$ 
@9    return  $A[n-1..n][pos-1..pos]$ 
@10 end if
@11  $pos \leftarrow \text{FLIPCOLINV}(A[1..n][1..n], 1)$ 
@12 return  $A[pos-1..pos][1..2]$ 

```

Твърдение 4.4.

При вход 4-интересна матрица $A[1..n][1..n]$, 4INTERESTING връща за време $O(\log n)$ 4-интересна 2×2 подматрица на A .

Най-съществената част от доказателството на коректността **Твърдение 4.4** е фактът, че ако **Q11** се достигне, то от **Лема 4.3** следва, че $A_{n1} > A_{11}$.
Времева сложност е $\mathcal{O}(\log n)$.

Твърдение 4.5 (Частни случаи на уравнение, породено от *Разделяй и владей*).
Ако $c \in \mathbb{R}^+$, $b > 1$, а $T : \mathbb{N} \rightarrow \mathbb{R}^+$ е функция, за която

$$T(n) \leq a \text{ при } n < b$$

$$T(n) \leq c T\left(\left\lceil \frac{n}{b} \right\rceil\right) + f(n) \quad , \text{ то}$$

- ако $f(n) \in \mathcal{O}(n^\alpha)$ с $\alpha < \log_b c$, то $T \in \mathcal{O}(n^{\log_b c})$
- ако $f(n) \in \Omega(n^\alpha)$ с $\alpha > \log_b c$, то $T \in \mathcal{O}(n^\alpha)$
- ако $f(n) \in \Theta(n^{\log_b c})$, то $T \in \mathcal{O}(n^{\log_b c} \log_b n)$

Доказателство. Развийте k пъти използвайки второто неравенство и изберете $k \leq \log_2 n \leq k + 1$. Заключете за асимптотиката на получената сума на две функции. $\square$

Примери:

- $T(n) \leq 9T\left(\frac{n}{3}\right) + \mathcal{O}(n) \rightsquigarrow T(n) \in \mathcal{O}(n^2)$
- $T(n) \leq 4T\left(\frac{n}{5}\right) + \mathcal{O}(n) \rightsquigarrow T(n) \in \mathcal{O}(n)$
- $T(n) \leq 4T\left(\frac{n}{5}\right) + \mathcal{O}(1) \rightsquigarrow T(n) \in \mathcal{O}(n^{\log_5 4})$
- $T(n) \leq T\left(\frac{n}{2}\right) + \mathcal{O}(1) \rightsquigarrow T(n) \in \mathcal{O}(\log n)$
- $T(n) \leq 2T\left(\frac{n}{2}\right) + \mathcal{O}(n) \rightsquigarrow T(n) \in \mathcal{O}(n \log n)$
- $T(n) \leq 3T\left(\frac{n}{2}\right) + \mathcal{O}(n) \rightsquigarrow T(n) \in \mathcal{O}(n^{\log_2 3})$
- $T(n) \leq 2T\left(\frac{n}{2}\right) + \mathcal{O}(n \log n)$ - **Твърдение 4.5** не е приложимо, решаваме с различни методи, например с развиване на рекурентната зависимост за $F(n) = \frac{T(n)}{n}$ следва, че $F(n) \in \mathcal{O}(\log^2 n)$

Задача 4.3 (EXPONENT).

Разглеждаме следния проблем, който ще решим използвайки стратегията *Разделяй и владей*:

Вход: $n \in \mathbb{Q}, k \in \mathbb{N}, m \in \mathbb{N}^+$

Изход: $n^k \pmod m$

EXPONENT(n : rational, k : natural, m : positive natural)

```

Q1  if  $k = 0$  then return 1
Q2   $h \leftarrow \left\lfloor \frac{k}{2} \right\rfloor$ 
Q3   $y \leftarrow \text{EXPONENT}(n, h, m)$ 
Q4   $y \leftarrow y * y \pmod m$ 
Q5  if  $k \equiv 1 \pmod 2$  then
```

```

@6    $y \leftarrow y * n \pmod m$ 
@7   end if
@8   return  $y$ 

```

Коректност показваме лесно с индукция по входа k .

Относно времевата сложност: всички извиквания на `EXPONENT` при вход k са $\log k$. Във всяко такова извикване се изпълняват операции за константно време, заедно с най-много 2 умножения на числа, подадени на вход, т.е.

$$Time_{Exponent} = \mathcal{O}(\log k) Time_{Multiply}(m)$$

където `MULTIPLY` е някакъв алгоритъм за умножение на два числа $\leq m$.

Възможни са следните три алгоритъма:

- Стандартен, познат от училищния курс, със сложност $\Theta((\log m)^2)$
- Алгоритъм на *Karatsuba* със сложност $\mathcal{O}((\log m)^{\log_2 3})$
- Алгоритъм, базиран на `FFT` за сложност $\mathcal{O}(\log m \log(\log m))$

Неизвестно е дали има алгоритъм със сложност $T \in \mathcal{O}(q \log q) \setminus \Theta(q \log q)$ ($= o(q \log q)$) за умножение на две q -битови числа.

Задача 4.4 (CLOSEST PAIR).

При наличие на n точки в равнината, колко е най-малкото разстояние между някои две от тях? Проблемът очевидно е приложим в практиката

Точка с рационални координати в равнината представяме със записа:

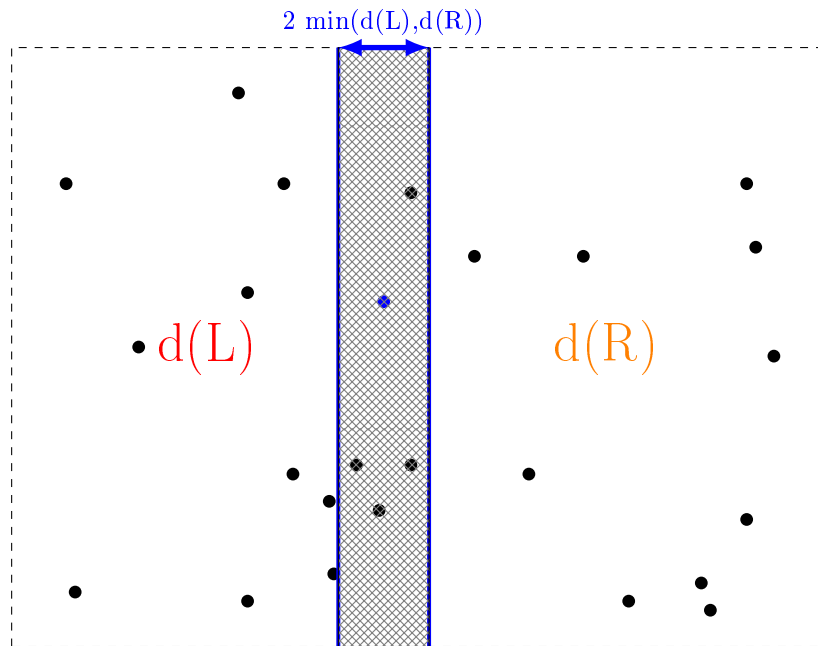
$$\text{Point} = \left\{ \begin{array}{l} x : \text{rational} \\ y : \text{rational} \end{array} \right\}$$

Вход: $A[1..n]$: масив от *Point* записи.

Изход: $\min \{d(A[i], A[j]) \mid 1 \leq i < j \leq n\}$.

Тривиалният алгоритъм, търсещ минимума по всички двойки различни точки за време $\Theta(n^2)$, ще наречем **CLOSESTPAIRDUMMY**.

Тук алгоритъм от вида *Разделяй и владей* е приложим.



Фигура 1: *Разделяй и владей* идея за най-близки точки в равнината

Лема 4.6. Ако $|p.x - q.x| > d$ или $|p.y - q.y| > d$, то $d(x, y) > d$.

Доказателство. Тривиално ограничение отдолу на евклидовото разстояние. □

Твърдение 4.7. Дефинираме за множество (масив) $A[l \dots r]$ от *Point*

$$d(A) = \min_{1 \leq i < j \leq r} d(A[i], A[j])$$

Ако $P'[l \dots r]$ са сортирани по поле x възходящо и $m = \lfloor \frac{l+r}{2} \rfloor$, дефинираме

$$\begin{aligned} L &:= P'[l \dots m-1] \\ R &:= P'[m \dots r] \\ S &:= \{P[i] \mid |P'[m-1].x - P[i].x| \leq \min\{d(L), d(R)\}\} \end{aligned}$$

Тогава $d(P) = \min\{d(L), d(R), d(S)\}$.

Доказателство. Сравнението $\leq$ следва от множествени съображения. Допускаме, че равенство няма и изпозваме **Твърдение 4.7** за двойката достигаща минимум отляво. $\square$

Сега $d(L)$ и $d(R)$ намираме рекурсивно ("разделяй"). Остава да намерим $d(S)$. Как да реализираме стъпката "владей" най-ефективно?

Пълно изчерпване за $\mathcal{O}(n^2)$ (в най-лошия случай почти всички точки са в ивицата) води до алгоритъм за $\mathcal{O}(n^2)$

Сортиране на средната ивица всеки път за $\mathcal{O}(n \log n)$ и обхождане отдолу нагоре на точките всеки път (защо...**Лема 4.9** след псевдокода) води до алгоритъм с времева сложност $\mathcal{O}(n \log^2 n)$. По-добра идея е да се възползваме от рекурсията, която да сортира вместо нас...

Нека **MERGEY** е модификация на **MERGE**, която модифицира масив от **Points**, който е конкатенация на два сортирани по поле y масива от **Points**, за време $\mathcal{O}(n)$, където n е големината на входа.

Нека също **INSERTIONSORTY** е модификация на **INSERTIONSORT**, която сортира масив от точки възходящо по поле y за време $\mathcal{O}(n^2)$.

CLOSESTPAIR($P[1 \dots n]$: array of Points)

```

01 MERGESORTPOINTS $X$ ( $P[1 \dots n]$ )
02 return CLOSESTPAIRREC( $P[1 \dots n]$ )

```

CLOSESTPAIRREC($P[l \dots r]$: array of Points)

```

01 if  $r - l < 3$  then
02   INSERTIONSORTY( $P[l \dots r]$ )
03   return CLOSESTPAIRDUMMY( $P[l \dots r]$ )
04 end if
05  $m \leftarrow \lfloor \frac{l+r}{2} \rfloor$ 
06  $midPoint \leftarrow P[m-1]$ 
07  $d1 \leftarrow CLOSESTPAIRREC(P[l \dots m-1])$ 
08  $d2 \leftarrow CLOSESTPAIRREC(P[m \dots r])$ 
09  $d \leftarrow \min\{d1, d2\}$ 
10  $d \leftarrow \min\{d, INSPECTMIDSTRIPE(P[l \dots r], m, midPoint, d)\}$ 
11 MERGEY( $P[l \dots r]$ ,  $m$ )
12 return  $d$ 

```

INSPECTMIDSTRIPE($P[l \dots r]$: Points, m : index, $midPoint$: Point, d : p. rational)

```

01  $S[1 \dots k] \leftarrow POINTSINSTRIPEMERGEDY(P[l \dots r], midPoint, d)$ 
02 for  $i = 1$  to  $k$  do
03    $j \leftarrow i - 1$ 
04   while  $j \geq 1 \wedge S[j].y > S[i].y - d$  do
05      $d = \min\{d, DISTANCE(S[i], S[j])\}$ 

```

```

@6   j ← j - 1
@7   end while
@8   end for
@9   return d

```

Твърдение 4.8. При вход масив от *Point* $P[l \dots r]$, "средна" точка $midPoint$ и рационално число d , $INSPECTMIDSTRIPE(P, m, midPoint, d)$ връща

$$\min \{d, d(S)\}$$

Доказателство. Директно приложение на **Лема 4.6** (и коректност на $POINTSINSTRIPEMERGEDY$). $\square$

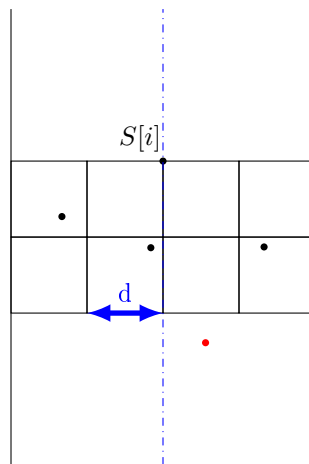
Коректността на $CLOSESTPAIRREC$ е индукция по броя на точките във входа с използване на **Твърдение 4.7** и **Твърдение 4.8**.

Относно времевата сложност, за $T(n) := Time_{ClosestPairRec}$:

$$\begin{aligned}
T(n) &\leq a \text{ при } n < 4 \\
T(n) &\leq 2T\left(\frac{n}{2}\right) + Time_{InspectMidStripe}(n)
\end{aligned}$$

Лема 4.9. Цикълът на ред @4 в $INSPECTMIDSTRIPE$ се изпълнява най-много 7 пъти.

Доказателство. Да допуснем, че има поне 8 точки с ордината $\geq s[i].y - d$. Тъй като няма две точки от една и съща страна на разстояние $\leq d$, то няма две точки в една клетка на Фигура 2. Тогава всички 8 точки попадат в различни клетки, т.е. някоя попада в една клетка с $s[i]$, което е невъзможно, поради същия аргумент, откъде противоречието с броя на точките следва. $\square$



Фигура 2: Илюстрация на **Лема 4.9**

Сега $Time_{InspectMidStripe}(n) = \mathcal{O}(n^2)$ и $CLOSESTPAIRREC$ поражда следната зависимост:

$$\begin{aligned}
T(n) &\leq a \text{ при } n < 4 \\
T(n) &\leq 2T\left(\frac{n}{2}\right) + \mathcal{O}(n)
\end{aligned}$$

тоест $Time_{ClosestPair}(n) = \mathcal{O}(n \log n) + T(n) = \mathcal{O}(n \log n)$.

Задача 4.5 (k -ти по големина елемент в два сортирани масива).

Популярна задача е намирането на медиана на два сортирани масива. Лесно се съобразява линейно решение: сливане на сортираните масиви или паралелно обхождане с два индекса.

Стратегия от вида *Разделяй и владей* води до значително по-добро решение.

Ще намерим решение на по-общия проблем:

Вход: $A[1...n], B[1...m]$ - масиви от рационални числа.

Изход: k -ти по големина елемент в сортираната пермутация на $A \circ B$.

Относно времевата сложност на **KTHMERGED**:

Полагаме $n := t - s + 1$, $m := q - p + 1$, $k := \lfloor \log n \rfloor + \lfloor \log m \rfloor$. Тогава за

$$T(k) := \max_{n+m=k} Time_{KthMerged}(2^n + 2^m)$$

имаме, че

$$T(k) \leq T(k-1) + \mathcal{O}(1)$$

т.е. $T(k) = \mathcal{O}(k)$.

Оттук $Time_{KthMerged}(n+m) \leq T(\lceil \log n \rceil + \lceil \log m \rceil) = \mathcal{O}(\lceil \log n \rceil + \lceil \log m \rceil)$.

Следващите твърдения използват наготово факта, че всеки алгоритъм, разрешаващ проблема за това дали масив съдържа само различни елементи, има сложност $\Omega(n \log n)$ за n елемента.

Твърдение 4.10 (долна граница за **CLOSEST PAIR**).

Ще покажем, че проблемът **CLOSEST PAIR** е $\Omega(n \log n)$, като посочим редукция на **UNIQUENESS** към **CLOSEST PAIR**.

$$\text{UNIQUENESS} \lll_n \text{CLOSEST PAIR}$$

Preprocessing:

1. За всеки елемент $A[i] \rightsquigarrow (A[i], 0)$
2. $A[i] = A[j]$ за някои $1 \leq i < j \leq n$ т.с.т.к. най-късото евклидово разстояние между две от точките е 0, т.е. връщаме дали **CLOSEST PAIR** при вход съпоставените точки е 0.

Твърдение 4.11 (долна граница за **MINIMUM ABSOLUTE SUBARRAY**).

$$\text{UNIQUENESS} \lll_n \text{MINIMUM ABSOLUTE SUBARRAY}$$

Алгоритъм за UNIQUENESS:

1. Създаваме масив $B[1...n-1]$, за който $B[i] = A[i+1] - A[i]$
2. Връщаме дали **MINIMUM ABOSULE SUBARRAY** при вход B е 0

Следствие: Всеки алгоритъм, разрешаващ **MINIMUM ABOSULE SUBARRAY** има времева сложност в най-лошия случай $\Omega(n \log n)$

(Следва от долната граница за **UNIQUENESS**, изложена като задача след лекция)

Задача 4.6 (3-EQUAL).

Вход: $A[1...n]$ - массив от рационални числа.

Изход: има ли $1 \leq i < j < k \leq n$, за които $A[i] = A[j] = A[k]$. Покажете, че

$$\text{UNIQUENESS} \lll_n \text{3-EQUAL}$$

Какво следствие можете да направите от тази задача?

Задача 4.7 (MODE).

Вход: $A[1...n]$ - массив от рационални числа.

Изход: $\arg \max_{1 \leq i \leq n} |\{j \mid 1 \leq j \leq n \ \& \ A[j] = A[i]\}|$. Покажете, че

$$\text{UNIQUENESS} \lll_n \text{MODE}$$

Какво следствие можете да направите от тази задача?

5 Алчни алгоритми

Няма формална дефиниция; алгоритъм, който избира локално най-доброто решение. Част от вече разгледаните задачи може да се разглеждат като алчни алгоритми.

Основни аргументи за коректност на "алчен" подход:

1. Локалното най-добро е и глобално най-добро (greedy stays ahead)
2. Аргумент на размяна (exchange argument)

Задача 5.1 (MAXIMUM STOCKS).

Разполагаме с k долара и искаме да закупим акции в период от n дни. Предварително е ясно, че на ден i цената на акция е $C[i]$, но има ограничение за закупуване само на $L[i]$ акции за този ден. Каква стратегия да следваме, за да закупим максимален брой акции.

Вход: $C[1..n]$ - масив от цени, рационални числа, $L[1..n]$ - лимити за количество акции в конкретен ден, k - неотрицателно рационално число.

Изход: $\max\{v \mid \text{можем да закупим } v \text{ акции за тези } n \text{ дни}\}$

BUYGREEDY($C[1..n] : \text{rationals}$, $L[1..n] : \text{array of naturals}$, $k : \text{non-negative rational}$)

```

01  SORTBYPRICESINCREASING(  $C[1..n]$ ,  $L[1..n]$  )
02   $stocks \leftarrow 0$ 
03  for  $i = 1$  to  $n$  do
04     $stocksToday \leftarrow \min\left\{\frac{k}{C[i]}, L[i]\right\}$ 
05     $stocks \leftarrow stocks + stocksToday$ 
06     $k \leftarrow k - stocksToday * C[i]$ 
07  end for
08  return  $stocks$ 

```

Задачата е по-известна като **FRACTIONAL KNAPSACK**. Относно коректността, формулирайте инвариант за цикъла на 03 и допуснете, че има разпределение, в което от i -тата по големина стока е взето количество $< \min\left\{\frac{k}{C[i]}, L[i]\right\}$.

Времевата сложност е $\mathcal{O}(n \log n)$.

Задача 5.2 (MAKE CHANGE).

В реалността типичният подход, когато очакваме ресто, е касиерът да взима най-голямата деноминация, докато не събере необходимата сума. Всъщност този подход строго зависи от монетната система, с която се работи. Историята казва, че деноминациите на първата валута в Римската империя са били:

деноминация	Denarius	Sesterius	Dupondius	As	Semis	Quincunx	Triens	Quadrans	Uncia
струва X Denarius	1	4	5	10	20	24	30	40	120

Вижда се, че алчният подход тогава не е работел дори за 8 Denarius.

Монетна система дефинираме като строго растяща редица от положителни цели числа $\{d_i\}_{i=1}^k$, за която $d_1 = 1$ (за да описва всички положителни цели стойности).

Казваме, че редицата $\{a_i\}_{i=1}^n$ е **ресто** за x , ако $a_i \in d$ за $1 \leq i \leq n$ и $\sum_{i=1}^n a_i = x$.

Кога "алчният" подход дава оптимален резултат?

Вход: $D[1..n]$: масив от деноминации, подредени в растящ ред, x - положително цяло число.
Изход: $\min\{n \mid \{a_i\}_{i=1}^n \text{ е ресто за } x\}$.

GREEDYCHANGE($D[1..k]$: increasing coin denominations, $change$: non-negative integer)

```

01  coin  $\leftarrow k$ 
02  count  $\leftarrow 0$ 
03  while  $change > 0$  do
04    while  $coin \geq 1 \wedge change < D[coin]$  do
05      coin  $\leftarrow coin - 1$ 
06    end while
07    change  $\leftarrow change - D[coin]$ 
08    count  $\leftarrow count + 1$ 
09  end while
10  return count

```

Твърдение 5.1. При вход масив от деноминации (положителни цели числа) $D[1..k]$, такива, че

$$1 = D[1] \mid D[2] \mid \dots \mid D[k]$$

и положително естествено число $change$, **GREEDYCHANGE** намира минималния брой монети в ресто за $change$.

Лема 5.2. Ако $\{a_i\}_{i=1}^n$ е минимално ресто за x и $a_i < D[j]$ за всяко $1 \leq i \leq n$ и някое $1 \leq j \leq k$, то $x \leq D[j] - 1$.

Твърдение 5.3. При всеки вход, времевата сложност на **GREEDYCHANGE** в най-лошия случай е $\mathcal{O}(x + k)$.

Задача 5.3 (INTERVAL SCHEDULING, Домашно 1, 2023г.).

Затрупан от домашни и контролни по време на семестъра, студентът Иван Иванов решил по време на Великденската си ваканция да се отдаде на безкраен купон. За съжаление и тук нещата не са толкова прости, колкото Иван си представял. Някои мероприятия се застъпват, а той би искал като отиде на едно мероприятие да бъде там от неговото начало до самия му край. Това, което му помага е, че все още си спомня някои полезни умения от средното си образование в Hagwardt, с които може да се телепортира. Студентът Иванов разполага със списък от n мероприятия. Всяко мероприятие има вида (i, s_i, e_i) , където $1 \leq i \leq n$ е уникален индекс на мероприятието, s_i задава началния, а e_i – крайния момент на мероприятието i . Да се докаже, че ако Иван иска да посети възможно най-много мероприятия, то:

1. може да премахне от своя списък всяко мероприятие (j, s_j, e_j) , за което има мероприятие (i, s_i, e_i) със свойството $(s_i, e_i) \subseteq (s_j, e_j)$.
2. може да постигне своята цел и като посети мероприятие, което започва възможно най-рано.
3. Да се предложи алгоритъм с времева сложност $\mathcal{O}(n \log n)$, който решава следния проблем:

Вход: $S[1..n], E[1..n]$, където $(S[i], E[i])$ е времевият интервал на мероприятието i

Изход: k - максимален брой мероприятия, които Иван Иванов може да посети.

Да се докаже коректността и времевата сложност на предложения алгоритъм.

MAXIMUMACTIVITIESREC($S[1..n]$: integers, $E[1..n]$: integers)

```

01  if  $n = 1$  then
02     $last \leftarrow 1$ 
03    return 1
04  end if
05   $(res, last) \leftarrow \text{MAXIMUMACTIVITIESREC}(S[1..n-1], E[1..n-1])$ 
06  if  $S[n] \geq E[last]$  then
07     $last \leftarrow n$ 
08    return  $(res + 1, n)$ 
09  end if
10  return  $(res, last)$ 

```

Твърдение 5.4. При вход представяне на множество $I = \{I_1, \dots, I_n\}$, което не съдържа интервали $I_j \subseteq I_k$, ACTIVITYSELECTION връща големината на максимална антиклика в G_I (интервалният граф с върхове I).

Доказателство. Използваме доказаното в 2., както и помощното твърдение, че за множество от интервали I и интервал (s_n, e_n) , такъв, че $e_n > e$ за всяко $(s, e) \in I$, е изпълнено, че

$$k(I \cup \{(s_n, e_n)\}) \leq k(I) + 1$$

□

Итеративният подход към този проблем е:

ACTIVITYSELECTION($S[1..n]$: integers, $E[1..n]$: integers)

```

01  MERGESORTBYEND( $S[1..n], E[1..n]$ )
02   $count \leftarrow 0$ 
03   $last\_end \leftarrow -\infty$ 
04  for  $i = 1$  to  $n$  do
05    if  $S[i] > last\_end$  then
06       $count \leftarrow count + 1$ 
07       $last\_end \leftarrow E[i]$ 
08    end if
09  end for
10  return  $count$ 

```

Задача 5.4 (MINIMUM HALLS).

Минимален брой зали за събития (хроматично число в интервален граф)

Вход: $S[1..n], E[1..n]$, където $(S[i], E[i])$ е времеви интервал на мероприятиято i

Изход: k - минимален брой зали за провеждането на събитията.

```

INTERVALCOLORING(  $L[1..n]$  : array of rationals,  $R[1..n]$  : array of rationals)
@1   $C[1..n] \leftarrow \text{malloc}(n)$ : ARRAY of naturals
@2   $A[1..2 * n] \leftarrow \text{malloc}(2 * n)$ : ARRAY of Endpoints
@3  for  $i = 1$  to  $n$  do
@4     $A[2 * i - 1] \leftarrow \text{Endpoint}(x \leftarrow L[i], \text{closing} \leftarrow \text{false}, \text{index} \leftarrow i)$ 
@5     $A[2 * i] \leftarrow \text{Endpoint}(x \leftarrow R[i], \text{closing} \leftarrow \text{true}, \text{index} \leftarrow i)$ 
@6  end for
@7  MERGESORTENDPOINTS( $A[1..2n]$ )
@8   $Q \leftarrow \text{QUEUE}()$ 
@9   $colors \leftarrow 0$ 
@10 for  $i = 1$  to  $2n$  do
@11   if  $A[i].\text{closing} = \text{true}$  then
@12     ENQUEUE( $Q, C[A[i].\text{index}]$ )
@13   else if  $Q = \emptyset$  then
@14      $colors \leftarrow colors + 1$ 
@15      $C[A[i].\text{index}] \leftarrow colors$ 
@16   else
@17      $C[A[i].\text{index}] \leftarrow \text{DEQUEUE}(Q)$ 
@18   end if
@19 end for
@20 return  $colors$ 

```

Твърдение 5.5. При вход представяне на множество $I = \{I_1, \dots, I_n\}$, INTERVALCOLORING връща хроматичното число на G_I (интервалният граф с върхове I).

Доказателство. Относно коректността, покажете, че за интервалния граф $G_I(I, E)$, където $\{I_i, I_j\}$ е ребро т.с.т.к. $I_i \cap I_j \neq \emptyset$ е изпълнено

$$\chi(G) = \omega(G)$$

(лявото е най-много върнатото на @20 $colors'$ (докажете, че алгоритъмът връща оцветяване в C чрез инвариант за опашката Q), а дясното е поне $colors'$ (защо е била увеличена за последно променливата $colors$?)) $\square$

Задача 5.5 (MINIMUM AVERAGE COMPLETION TIME).

Дадено е множество от n задачи, като задача i се нуждае от $P[i]$ единици време, за да бъде изпълнена. Разполагаме само с една машина, която може да изпълнява най-много една задача в даден момент. График на задачите ще наричаме пермутация π на $\{1, \dots, n\}$, даваща реда на изпълнение на задачите от машината.

Средно време на завършване на π наричаме

$$c(\pi) = \frac{1}{n} \sum_{i=1}^n c_{\pi(i)}$$

където $c_{\pi(i)}$ е времето на завършване на задача i .

С $c^*(P)$ бележим минималното средно време на завършване по всички графици за задачите

$P[1...n]$.

Да се предложи алгоритъм с времева сложност $\mathcal{O}(n \log n)$, който решава следния проблем:

Вход: $P[1...n]$ - времена за изпълнение.

Изход: $c^*(P)$

Лема 5.6 (Permutational inequality). Нека $\{a_i\}_{i=1}^n$ и $\{b_i\}_{i=1}^n$ са такива, че b е монотонно намаляваща, а a е монотонно растяща. Тогава за всяка пермутация $\sigma \in S_n$

$$\sum_{i=1}^n a_i b_i \leq \sum_{i=1}^n a_{\sigma(i)} b_i$$

Доказателство. Разгледайте пермутацията, минимизираща $\sum_{i=1}^n a_{\sigma(i)} b_i$ с най-малко двойки индекси (i, j) , за които $i < j$ и $a_{\sigma(i)} > a_{\sigma(j)}$. Разменете такава двойка индекси и достигнете до противоречие с избора на пермутацията. $\square$

Задача 5.6 (MINIMUM SCHEDULING PENALTY (Матроиди)).

Поглед върху матроиди в контекста на Activity Scheduling

[CLRS, 16.5, страница 443 по номерацията, въведена в документа](#)

Отново разполагаме със задачи $p_1, \dots, p_n$, всяка от които се извършва за единица време. Представени са като наредени двойки (d_i, ω_i) , съответно краен срок за изпълнение на задачата и глоба за неспазването му. За сроковете имаме, че $1 \leq d_i \leq n$ и $\omega_i \geq 0$

Формално, график наричаме пермутация π на задачите. Глобата на графика π наричаме:

$$\omega(\pi) = \sum_{i=1}^n c(p_{\pi(i)})$$

където

$$c(p_{\pi(i)}) = \begin{cases} 0 & , \text{ако } i \leq d_{\pi(i)} \\ \omega_{\pi(i)} & \text{иначе} \end{cases}$$

За задачите $p_1, \dots, p_n$ оптимален график наричаме график π^* , който минимизира $\omega(\pi)$ по всички графици, и $\omega^*(\{p_1, \dots, p_n\}) = \omega(\pi^*)$.

Вход: $D[1...n], W[1...n]$ - крайни срокове и глоби на множество от задачи P .

Изход: $\omega^*(P)$

Ще казваме, че график е в *нормална* форма, ако всички незакъсняващи задачи са преди закъсняващите.

Твърдение 5.7. Да се докаже, че за всеки график π има график в нормална форма π' , такъв, че

$$\omega(\pi') \leq \omega(\pi)$$

Ще казваме, че график е в *канонична* форма, ако е в нормална форма и незакъсняващите задачи са подредени в нарастващ ред по крайния срок.

Твърдение 5.8. Да се докаже, че за всеки график π има график в канонична форма π'' , такъв, че

$$\omega(\pi'') \leq \omega(\pi)$$

Сега задачата се свежда до определяне на задачите, които да завършат навреме.

Множество от задачи ще наричаме *изпълнимо*, ако има за него има график π , такъв, че няма закъсняващи задачи.

Да означим с $\mathcal{E}$ подмножествата на P , които са изпълними. Ще покажем, че $(P, \mathcal{E})$ е матроид. За целта първо търсим критерий за лесна проверка дали множество от задачи е изпълнимо. Той следва от следната лема:

Лема 5.9. *За всяко множество A от задачи следните са еквивалентни:*

- A е изпълнимо
- $N_t(A) \leq t$ за $t = 0, \dots, n$, където $N_t(A)$ е броят на задачите в A , чиито краен срок е $\leq t$
- графикът, при който задачите в A са наредени по нарастване на крайния срок, не съдържа закъсняващи задачи

Сега втората точка на **Лема 5.9** ни дава НДУ за проверка дали множество от задачи е изпълнимо за $|\mathcal{A}|$ време, например така:

ISINDEPENDENT($D[1\dots n]$: deadlines, $W[1\dots n]$: penalties, $I[1\dots k]$: indices of the subset)

```

01   $Nt \leftarrow \text{malloc}(k, 0)$ : ARRAY of integers
02  for  $i = 1$  to  $k$  do
03    if  $D[I[i]] \geq k$  then
04       $Nt[k] \leftarrow Nt[k] + 1$ 
05    else
06       $Nt[D[I[i]]] \leftarrow Nt[D[I[i]]] + 1$ 
07    end if
08  end for
09  for  $i = 1$  to  $n$  do
10     $Nt[i] \leftarrow Nt[i] + Nt[i - 1]$ 
11  end for
12  for  $i = 1$  to  $n$  do
13    if  $Nt[i] > i$  then return false
14  end for
15  return true

```

Лема 5.10. $(P, \mathcal{E})$ е матроид.

Сега съобразявайки, че минимизирането на глобата от закъсняващи задачи съответства на максимизиране на глобата от незакъсняващи задачи, можем да приложим алчен алгоритъм върху матроид, т.е. сортираме задачите в намаляване на глобата и взимаме задача, само ако не разваля "изпълняемостта" .

Времева сложност е $\mathcal{O}(n^2)$.

6 Графи

Основни представяния на граф $G = (V, E)$:

- матрица на съседство: матрица $A = (a_{ij})_{i,j=1}^{|V|,|V|} \in \mathbb{M}_{0,1}$, където

$$a_{ij} = 1 \iff (v_i, v_j) \in E$$

- списък от ребра: списък, представящ E ;
- списъци на съседство: за всеки връх $v \in V$ пазим списък, представящ $N(v)$ (множеството от съседите на v в G);
- канонично представяне: масиви $V[1...n]$ и $E[1...m]$, в които съседите на i са върховете $E[V[i]]...E[V[i+1]-1]$.

Всяко представяне може да се сведе до друго за време $\mathcal{O}(|V|^2)$.

Задача 6.1 (TRIANGLE). По даден граф G , да се провери дали G съдържа триъгълник (цикъл с дължина 3).

Вход: $G(V, E)$ - неориентиран граф.

Изход: $|\{c = (v, u, w) \mid c \text{ е цикъл в } G\}|$

В зависимост от необходимостта, различни алгоритми с различна оценка на времевата им сложност могат да бъдат използвани:

- $\mathcal{O}(|V|^3)$ (представяне с матрица)

TRIPLETS($G(V, E)$: undirected graph)

```

01  triangles ← 0
02  for v ∈ V do
03    for u ∈ V do
04      for w ∈ V do
05        if (v, u) ∈ E ∧ (u, w) ∈ E ∧ (w, v) ∈ E then
06          triangles ← triangles + 1
07        end if
08      end for
09    end for
10  end for
11  return triangles/6
```

- $\mathcal{O}(|V||E|)$ (списъци на съседство)

- $\mathcal{O}(\sum_{v \in V} d(v)^2)$

PAIRNEIGHBOURS($G(V, E)$: undirected graph)

```

01  triangles ← 0
02  for v ∈ V do
03    for u ∈ N(v) do
04      for w ∈ N(v) do
05        if (u, w) ∈ E then
06          triangles ← triangles + 1
07        return triangles/6
```

- $\mathcal{O}(|E|\sqrt{|E|})$

Можем да избегнем повторното броене на върхове ако "фиксираме" само "най-малкия" от тях по релация

$$u \prec v \iff d(u) < d(v) \vee (d(u) = d(v) \wedge u \lesssim v)$$

където $\lesssim$ е някаква линейна наредба на върховете (лексикографска наредба е разумен вариант)

PAIRNEIGHBOURSLowDEGREE($G(V, E)$: *undirected graph*)

```

01  triangles ← 0
02  for v ∈ V do
03    for u ∈ N(v) ∩ {u ≻ v} do
04      for w ∈ N(v) ∩ {u ≻ v} do
05        if (u, w) ∈ E then
06          triangles ← triangles + 1
07  return triangles

```

Коректността на **PAIRNEIGHBOURSLowDEGREE** е ясна. Относно времевата сложност, наблюдаваме, че върховете от висока степен извършват сравнително по-малка работа от тези от ниска степен. Дефинираме

$$V_{low} = \{v \in V \mid d(v) < \sqrt{|E|}\}$$

$$V_{high} = \{v \in V \mid d(v) \geq \sqrt{|E|}\}$$

Имаме, че $|V_{high}| \leq 2\sqrt{m}$ следва от $\sum_{v \in V_{high}} d(v) \leq \sum_{v \in V} d(v) = 2|E|$.

Ако на 02 $v \in V_{high}$, то всеки връх u, w , разгледани съответно на 03 и 04 също са от V_{high} . Тогава телата(, изпълнени на 02 с върхове от V_{high} , се изпълняват за време

$$\mathcal{O}(|V_{high}|^3) = \mathcal{O}\left(\left(2\sqrt{|E|}\right)^3\right) = \mathcal{O}(|E|\sqrt{|E|})$$

Относно телата с върхове от V_{low} , времевата сложност е (грубо ограничена отгоре)

$$\mathcal{O}\left(\sum_{v \in V_{low}} d(v)^2\right)$$

Ограничение отгоре за тази сума получаваме от следната **Задача 6.1**:

Лема 6.1. Нека $\{x_i\}_{i=1}^k$ с $k \in \mathbb{N}$ са положителни естествени числа, за които

$$\sum_{i=1}^k x_i \leq 2m$$

$$x_i < \sqrt{m} \quad \text{за } i = 1, \dots, k$$

Тогава

$$\sum_{i=1}^k x_i^2 < 4m\sqrt{m}$$

Доказателство. Нека $\{x_i\}_{i=1}^k$ е редица, максимизираща $\sum_{i=1}^k x_i^2$ с възможно най-малко k . Да допуснем, че има $1 \leq i < j \leq k$, такива, че

$$x_i + x_j < \sqrt{m}$$

Тогава имаме, че

$$x_i^2 + x_j^2 < (x_i + x_j)^2$$

Тоест заменянето на x_i и x_j със сбора им води до противоречие с избора на редицата. Следователно за произволни $1 \leq i < j \leq k$ е в сила, че

$$x_i + x_j \geq \sqrt{m}$$

Сега сумирайки по всевъзможните двойки $1 \leq i < j \leq k$ получаваме:

$$(k-1)2m \geq (k-1) \sum_{i=1}^n x_i \geq \binom{k}{2} \sqrt{m}$$

Оттук $k \leq 4\sqrt{m}$ и тогава

$$\sum_{i=1}^k x_i^2 < 4m\sqrt{m}$$

□

Обратно, от следва, че времевата сложност за телата с върхове $v \in V_{low}$ е $\mathcal{O}(|E|\sqrt{|E|})$ и общата такава е отново $\mathcal{O}(|E|\sqrt{|E|})$.

Тази граница е стабилна, т.е. алгоритъмът е с времева сложност $\Omega(|E|\sqrt{|E|})$, например при вход пълен граф...

- $\mathcal{O}(|V|^{\log_2 7})$ (матрица на съседство) Известен факт е, че ако $A = (a_{ij})_{i,j=1}^{|V|,|V|} \in \mathbb{M}_{0,1}$ представя граф G и $B = A^k$, то b_{ij} е броят пътища с дължина k от i до j .

Тоест е достатъчно да намерим сумата на главния диагонал на A^3 и да разделим на 6.

Задача 6.2 (Хамилтонов път в турнир).

Турнир наричаме ориентиран граф, в който е изпълнено

$$\forall u \in V \forall v \in V ((u, v) \in E \oplus (v, u) \in E)$$

Известен факт е, че всеки турнир съдържа Хамилтонов цикъл (защо?)

Вход: $G(V, E)$ - представяне на турнир.

Изход: $H[1..n]$ - Хамилтонов път в G .

Разглеждаме рекурсивен алгоритъм, който "вмъква" връх във вече построен Хамилтонов път от останалите върхове.

HAMILTONIANPATH($G(V, E)$: tournament)

① $H[1..n-1] \leftarrow \text{HAMILTONIANPATH}(G - v)$

② $\text{INSERTVERTEX}(G, H[1..n-1], n)$

INSERTVERTEX($G(V, E)$: tournament, $H[1..k]$: path in G , v : vertex in G)

① if $A[v][H[1]] = 1$ then

② return $v \circ H[1..k]$

③ if $A[H[k]][v] = 1$ then

④ return $H[1..k] \circ v$

⑤ $i \leftarrow 1$

⑥ while $i < k \wedge \neg(A[H[i]][v] = 1 \wedge A[v][H[i+1]] = 1)$ do

```

07    $i \leftarrow i + 1$ 
08   end while
09   return  $H[1...i] \circ v \circ H[i + 1...k]$ 

```

Задача 6.3 (Второ най-леко покриващо дърво).

Нека $G(V, E)$ е граф и $c : E \rightarrow \mathbb{R}^+$ е инекция. Търсим второто най-леко MST T^{**} на G .

Вход: $G = (\{1, \dots, n\}, E)$ - неориентиран граф,
 $C[1...n][1...n]$ - представяне на инекция $c : E \rightarrow \mathbb{R}^+$
 Изход: T^{**}

Лема 6.2. Нека $G = (V, E)$ е свързан граф и $c : E \rightarrow \mathbb{R}^+$ е инекция. Тогава G има единствено минимално покриващо дърво.

Доказателство. Допускаме, че има $T_1 \neq T_2$ - МПД-та за G . Тогава имаме, че $E_1 \Delta E_2 \neq \emptyset$ и нека $e \in E_1 \Delta E_2$ е с минимално тегло по c . Б.О.О. считаме, че $e \in E_1$.

Тогава $T_2 + e$ съдържа цикъл π . Сега в π има ребро $e' \in E_2 \setminus E_1$ (защото π не може да е цикъл от ребра от E_1).

Имаме, че $c(e) < c(e')$ (равенство не може да има от инективността на c). Сега за дървото $T' := T_2 - e + e'$ имаме, че е покриващо с $c(T') < c(T_2)$, което е противоречие с факта, че T_2 е МПД. $\square$

Лема 6.3. Нека $G = (V, E)$ е граф, $c : E \rightarrow \mathbb{R}^+$ е инекция и T^* е MST за G . Тогава

$$\exists e \in E^* \exists e' \in E \setminus E^* (T' = T^* - \{e\} + \{e'\} \text{ е второ най-леко покриващо дърво на } G)$$

Доказателство. Нека T' е второ най-леко покриващо дърво и $e \in E^* \setminus E'$. Тогава $T' + e$ съдържа цикъл π , който съдържа ребро $e' \in E' \setminus E^*$.

Сега имаме, че $c(e') > c(e)$ (неравенство $<$ води до дърво $T^* - e + e'$, по-леко от МПД-то).

Накрая, за покриващото дърво $T'' := T' - e' + e$ имаме, че

$$c(T'') < c(T')$$

тоест T'' е минимално покриващо дърво, но то е единствено по **Лема 6.2**, т.е. $T'' = T^* = T' - e' + e$ $\square$

Следствие: ако знаем най-тежкото ребро в пътя в MST на G между всеки два върха

$$(u, v) \in V \times V \mapsto w(x, y)$$

то разликата в теглата и необходимо ребро за смяна се задава съответно от

$$\min_{(x,y) \in E} \{c(x, y) - w(x, y)\}$$

$$\arg \min_{(x,y) \in E} \{c(x, y) - w(x, y)\}$$

SECONDMST($V[1...n], E[1...2m]$: undirected graph, $C[1...m]$: array of rationals)

```

01    $P[1...n] \leftarrow \text{KRUSKAL}(V[1...n], E[1...2m])$ 

```

```

@2  VT[1...n], ET[1...2n - 2], CT[1...2n - 2] ← CANONICALREPRESENTATION(P[1...n])
@3  DT[1...n][1...n] ← malloc(n2): ARRAY of rationals
@4  for i = 1 to n do
@5    DFSDISTANCE(VT[1...n], ET[1...2n - 2], CT[1...2n - 2], DT[1...n][1...n], i)
@6  end for
@7  (minDiff, minX, minY) ← (∞, 1, 1)
@8  for i = 1 to n do
@9    j ← V[i]
@10   while j ≤ m & j < V[i + 1] do
@11     if minDiff > C[i][E[j]] - DT[i][E[j]] then
@12       minDiff ← C[i][E[j]] - DT[i][E[j]]
@13       minX ← i
@14       minY ← E[j]
@15     end if
@16   end while
@17 end for
@18 return TOTALCOST(ET[1...2n - 2], CT[1...2n - 2]) + minDiff

```

Коректността на **SECONDMST** следва от изложената **Лема 6.3**, както и следствието от нея.
 Времева сложност: $\mathcal{O}(|V|^2)$.

Задача 6.4 (LONGEST TREE PATH).

Търсенето на най-дълъг прост път в граф е NP-трудна задача. Когато обаче графът е дърво, може лесно да се намери.

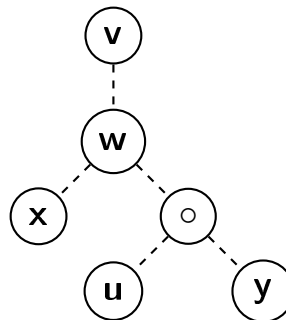
Вход: $V[1...n], E[1...2n - 2]$ - канонично представяне на дърво $T = (V, E)$.

Изход: $\max \{|\pi| \mid \pi \text{ е прост път в } T\}$

Лема 6.4. Нека $T = (V, E)$ е дърво и $v \in V$ е произволен. Тогава всеки връх u , който е край на най-дълъг път с начало v , е начало на най-дълъг път в T .

Доказателство. Разглеждаме T като кореново дърво T' с корен v . Нека u е някой край на най-дълъг прост път в T (респективно и в T'). Нека x и y са начало и край на някой най-дълъг прост път в T , а w е пресечната точка на пътищата от v до x , от v до y и от x до y . Разглеждаме следните 2 случая:

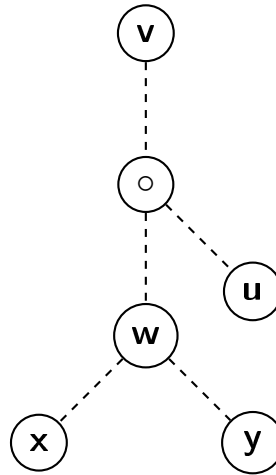
1. $u \in T'_w$



$$d(x, y) = d(x, w) + d(w, y) \leq d(x, w) + d(w, u) = d(x, u)$$

т.е. v е начало на най-дълъг път в T .

2. $u \notin T'_w$



Последователно имаме:

$$d(v, x) \leq d(v, u) \leq d(v, w) + d(w, u)$$

$$d(v, y) \leq d(v, u) \leq d(v, w) + d(w, u)$$

$$d(x, y) = d(x, w) + d(w, y) = d(v, x) + d(v, y) - 2d(v, w) \leq 2d(w, u)$$

$$\text{Сега } 2d(x, y) \leq d(x, y) + 2d(w, u) = d(x, w) + d(w, u) + d(u, w) + d(w, y) = d(u, x) + d(u, y)$$

$$\text{Следователно } \max \{d(u, x), d(u, y)\} \geq d(s, t)$$

т.е. v е начало на най-дълъг път в T .

□

Идея: Изпълняваме BFS върху T от произволен връх $v \in V$, с което намираме най-отдалечен от v връх u . След това изпълняваме BFS върху T от u , като дължината на най-дълъг път с начало u в T е дължината на най-дългия път с начало u . Коректността следва от **Лема 6.4**, а времевата сложност е

$$2\mathcal{O}(|V| + |E|) = |\mathcal{V}|$$

Задача 6.5 (MINIMUM TURNS).

Прав граф ще наричаме свързан ориентиран граф $G = (V, E)$, такъв, че $V \subset \mathbb{Z}$ е крайно, а $E = \{((x_1, y_1), (x_2, y_2)) \mid |x_1 - x_2| + |y_1 - y_2| = 1\} \cap V$.

За път $\pi = ((x_0, y_0), \dots, (x_k, y_k))$, брой на завоите в π ще наричаме

$$\tau(\pi) := k - 1 - |\{i \mid 0 < i < k \ \& \ x_i = (x_{i-1} + x_{i+1})/2 \ \& \ y_i = (y_{i-1} + y_{i+1})/2\}|$$

Търсим път между 2 върха в *прав граф*, който минимизира "завоите".

Вход: $V[1..n], E[1..m]$ - *прав граф*, u, v - *върхове*.

Изход: $P[1..k]$ - *път от u до v , минимизиращ завоите*.

Нека $G = (V, E)$ е *прав граф*. Дефинираме графът $G' = (V', E')$:

$$V' = V \times \{n, w, s, e\}$$

Нека $dir : E \rightarrow \{n, w, s, e\}$ е дефинирана като:

$$dir((x_1, y_1), (x_2, y_2)) = \begin{cases} n & \text{ако } x_2 = x_1 + 1 \text{ и } y_2 = y_1 \\ s & \text{ако } x_2 = x_1 - 1 \text{ и } y_2 = y_1 \\ w & \text{ако } x_2 = x_1 \text{ и } y_2 = y_1 - 1 \\ e & \text{ако } x_2 = x_1 \text{ и } y_2 = y_1 + 1 \end{cases}$$

$$E' = \{(\langle u, n \rangle, \langle v, dir(u, v) \rangle), (\langle u, w \rangle, \langle v, dir(u, v) \rangle), (\langle u, s \rangle, \langle v, dir(u, v) \rangle), (\langle u, e \rangle, \langle v, dir(u, v) \rangle) \mid (u, v) \in E\}$$

Въвеждаме и тегловна функция $c : E' \rightarrow \mathbb{Q}^+$, дефинирана като:

$$c(\langle u, d_1 \rangle, \langle v, d_2 \rangle) = \begin{cases} 1 & \text{ако } d_1 \neq d_2 \\ 0 & \text{ако } d_1 = d_2 \end{cases}$$

Имаме, че $|V'| = 4|V|$, както и $|E'| = 4|E|$.

Твърдение 6.5. *Пътят π е път с най-малко завои в G от u до v т.с.т.к. $c(\pi)$ е теглото на най-лек път в G' от някой от $\{u\} \times \{n, s, w, e\}$ до някой от $\{v\} \times \{n, w, s, e\}$.*

Използвайки алгоритъма на *Dijkstra*, задачата може да се реши за очаквано време $\mathcal{O}(|E| + |V| \log |V|)$. Тук обаче няма необходимост от сложна структура (като пирамида)... необходима е само опашка. Алгоритъмът е познат като 0/1 BFS.

MINTURNS($V[1...n], E[1...m] : \text{graph}, \quad C[1...m] : \text{pos. rat.}, \quad r : \text{vert.}, \quad p[1...n] : \text{parent f.}$)

```

01   $D[1...n] \leftarrow \text{malloc}(n, \infty)$ : ARRAY of rationals
02   $Q \leftarrow \text{QUEUE}()$ 
03   $D[r] \leftarrow 0$ 
04   $\text{ENQUEUEBACK}(Q, r)$ 
05  for  $i = 1$  to  $n - 1$  do
06     $v \leftarrow \text{DEQUEUE}(Q)$ 
07    for  $e = V[v]$  to  $\min\{m, V[r + 1] - 1\}$  do
08       $u \leftarrow E[e]$ 
09      if  $D[v] + C[u] < D[u]$  then
10         $D[u] \leftarrow D[v] + C[u]$ 
11         $p[u] \leftarrow v$ 
12        if  $C[u] = 1$  then
13           $\text{ENQUEUEBACK}(Q, u)$ 
14        else
15           $\text{ENQUEUEFRONT}(Q, u)$ 
16        end if
17      end if
18    end for
19  end for
```

Задача 6.6 (k-Clustering).

Нека $P = \{V_1, \dots, V_k\}$ е разбиване на върховете на граф $G(V, E)$. Отдалечаване на разбиването P ще наричаме

$$sp(P) = \min_{1 \leq i < j \leq k} \min_{e \in E \cap C_i \times C_j} c(e)$$

Търсим кое разбиване максимизира това отдалечаване, т.е. решение на следния проблем:

Вход: $V[1...n], E[1...2m]$ - неориентиран граф G , $C[1...2m]$ - представяне на теглова функция $c: E \rightarrow \mathbb{Q}$, k - брой компоненти на свързаност ($1 \leq k \leq n$)

Изход: d - максимално отдалечаване на разбиване на G на k компоненти.

Лема 6.6. Максимално разстояние между компонентите се достига при премахване на $k - 1$ -те най-тежки ребра от минимално покриващо дърво.

Доказателство. Нека $P^* := \{C_1, \dots, C_k\}$ е разбиването на върховете, получено след премахването на $(k - 1)$ -те най-тежки ребра от МПД.

Нека $P = \{D_1, \dots, D_k\}$ е произволно различно разбиване.

Сега имаме, че $sp(P^*) = c(e_{k-1})$, където e_{k-1} е $(k - 1)$ -вото най-тежко ребро в МПД.

Тъй като $P \not\sim P^*$, то има C_i , което не е подмножество на никое D_j , $1 \leq j \leq k$. Тогава има върхове $v_1, v_2 \in C_i$, които са в различни класове по P .

Нека $v_1 \in D_k, v_2 \in D_l, k \neq l$.

Сега v_1, v_2 са в една компонента, построена от алгоритъма на KRUSKAL, т.е. има път от v_1 до v_2 в C_i .

Нека в пътя от v_1 до v_2 , (u_1, u_2) е първото ребро, такова, че $u_1 \in D_k$ и $u_2 \notin D_k$. Тогава

$$sp(P) \leq c(u_1, u_2) \leq c(e_{k-1}) = sp(P^*)$$

Тоест няма разбиване с голямо отдалечаване от това, което алгоритъмът на KRUSKAL би построил. $\square$

Ребро с тегло моделираме със запис:

```
Edge = {
    source : integer
    target : integer
    cost : rational
}
```

CLUSTERING($V[1...n], E[1...2m]$: graph representation, k : number of components)

```

01  Edges  $\leftarrow$  malloc( $m$ ): ARRAY of Edges
02  FILLEDGES(Edges, V, E, C)
03  MERGESORTEDGESBYCOST(Edges[1... $m$ ])
04  components  $\leftarrow$   $n$ 
05  UF  $\leftarrow$  INITIALIZEUNIONFIND( $\{1, \dots, n\}$ )
06   $i \leftarrow 1$ 
07  while  $i \leq m$  & components  $> k$  do
08    if FIND(UF, Edges[ $i$ ].source)  $\neq$  FIND(UF, Edges[ $i$ ].target) then
09      UNION(UF, Edges[ $i$ ].source, Edges[ $i$ ].target)
10    components  $\leftarrow$  components - 1
11  end if
12   $i \leftarrow i + 1$ 
13  end while
14  if  $i > m$  then return  $\infty$ 
15  return Edges[ $i$ ].cost
```


Времева сложност: $\mathcal{O}(|E| \log |E| + k \log^* |V|) = \mathcal{O}(|E| \log |E|)$

Задача 6.7 (DIRECTED WIDEST PATH).

Целта ни е да максимизираме най-лекото ребро от всеки път от връх до друг връх в ориентиран граф.

Разглеждаме градове, между които са прекарани еднопосочни канали, всеки с известна дълбочина. Каква е максималната дълбочина на кораб, който може да стигне по каналите от град X до град Y ?

Вход: G - ориентиран граф, X, Y - върхове в G .

Изход: $\max \{ \min_{e \in \pi} c(e) \mid \pi \text{ е път от } X \text{ до } Y \text{ в } G \}$ - най-леко ребро в пътя с най-тежко такова

LESS(u, v : integers, $D[]$: context)

01 return $W[v] > W[u]$

MAXDEPTHSHIP($V[1..n], E[1..m]$: graph, $C[1..m]$: pos. rat., r : vert., $p[1..n]$: parent f.)

01 $flags[1..n] \leftarrow \text{malloc}(n)$: ARRAY of booleans

02 $D[1..n] \leftarrow \text{malloc}(n)$: ARRAY of rationals

03 $H \leftarrow \text{INITIALISEFIBONACCIHEAP}(n, less)$

04 for $i = V[r]$ to $\min \{m, V[r+1] - 1\}$ do

05 $W[v] \leftarrow C[E[i]]$

06 end for

07 for $v = 1$ to n do

08 if $v \neq r$ then

09 $flags[v] \leftarrow false$

010 if $p[v] \neq r$ then

011 $W[v] \leftarrow \infty$

012 $p[v] \leftarrow -1$

013 INSERTELEMENT(H, v, D)

014 end if

015 end if

016 end for

017 for $i = 1$ to $n - 1$ do

018 $v \leftarrow \text{GETMIN}(H)$

019 $flags[v] \leftarrow true$

020 EXTRACTMIN(H, D)

021 for $e = V[v]$ to $\min \{m, V[r+1] - 1\}$ do

022 $u \leftarrow E[e]$

023 if $flags[u] = false \ \& \ W[u] > C[E[e]]$ then

024 $W[u] \leftarrow C[E[e]]$

025 $p[u] \leftarrow v$

026 DECREASEKEY(H, D)

027 end if

028 end for

029 end for

Коректността се дължи на свойството, че алгоритъмът на DIJKSTRA разчита на структура

полупръстен, примери за която са:

- $(\mathbb{R}, +, \cdot)$ (класически пример за пръстен, в частност полупръстен)
- $(\mathbb{R}, \min, +)$ (най-евтин път при цена на път - сума)
- $(\mathbb{R}, \max, \min)$ (в тази задача)

Задача 6.8 (UNDIRECTED WIDEST PATH). Разглеждаме **Задача 6.7**, но за неориентиран граф, т.е. каналите вече са двупосочни. Оказва се, че тогава има и по-добър подход, с който лесно могат да се намерят най-широките пътища между всеки 2 върха.

Вход: $V[1...n], E[1...2m]$ - неориентиран граф.

Изход: $B[1...n][1...n]$ - масив от цени, където $B[u][v]$ е цената на най-лекото ребро в пътя от u до v с най-тежко такова.

Лема 6.7. Всички пътища в максимално покриващо дърво на неориентиран граф G са най-широки пътища (максимизиращи най-лекото ребро между началото и края си)

Хубаво следствие от **Лема 6.7** е следното:

Твърдение 6.8 (Най-тесни пътища). Всички пътища в минимално покриващо дърво на неориентиран граф G са най-тесни пътища (минимизиращи най-тежкото ребро между началото и края си)

Подходът към **UNDIRECTED WIDEST PATH** се основава на **Лема 6.7**

7 Динамично програмиране

Известно е, че всяка частична наредба може да се разшири до линейна. Идеята е известна още като "топологическо сортиране". Още нещо, ако транзитивното затваряне на антисиметрична релация е отново антисиметрично, то тя също може да се разшири до линейна наредба.

Динамичното програмиране в същността си е "просто" едно обхождане на топологически сортираните върхове на DAG и пресмятане на функция (на предшествениците им) във върховете. Трудността идва единствено при конструирането на графа.

Задача 7.1 (ODD COST DAG). Колко са пътищата в тегловен DAG от връх s до връх t , които имат нечетно тегло?

Вход: $V[1...n], E[1...m]$ - DAG, $C[1...m]$ - представяне на функция $c: E \rightarrow \mathbb{Z}$, $s, t \in \{1, \dots, n\}$.

Изход: $|\pi \mid \pi \text{ е път от } s \text{ до } t \ \& \ c(\pi) \equiv 1 \pmod{2}|$.

Нека

$$r_{v,x} \text{ е броят пътища от } s \text{ до } v \text{ с } \begin{cases} \text{нечетно тегло} & , \text{ ако } x = 0 \\ \text{четно тегло} & , \text{ иначе} \end{cases}$$

Доказваме, че

$$r_{v,x} = \sum_{(u,v) \in E} r_{u, (c(u,v)+x) \pmod{2}}$$

Търсеният отговор е $r_{t,1}$.

Задача 7.2 (SHORTEST PATHS COUNT).

Целта е да преброим колко са най-леките пътища между дадени 2 върха в граф?

Нека $d(u, v)$ е теглото на най-къс път от u до v .

Вход: $V[1...n], E[1...m]$ - канонично пр. на граф G , $C[1...m]$ - представяне на теглова функция $c: E \rightarrow \mathbb{Q}^+$, s, t - върхове.

Изход: $|\{\pi \mid \pi \text{ е път от } s \text{ до } t \text{ и } c(\pi) = d(s, t)\}|$

Ще наричаме реброто (u, v) *помощно*, ако $d(s, u) + c(u, v) = d(s, v)$.

Лема 7.1. Ако $\pi = (s, v_1, \dots, v_k, v)$ е най-лек път от s до v , то всяко от ребрата в π е *помощно*.

Тогава (от Лема 7.1) е достатъчно да търсим най-къси пътища само в графа $G' = (V, E')$, където $E' = \{e \mid e \in E \ \& \ e \text{ е помощно}\}$. След изпълнение на алгоритъм на *Dijkstra* за време $\mathcal{O}(|E| + |V| \log |V|)$, лесно построяваме G' .

Твърдение 7.2. Графът G' е ацикличен. Нещо повече, пътищата в G' са точно най-късите пътища в G .

От Твърдение 7.2 следва, че можем да приложим техниката на динамичното програмиране, за да преброим пътищата от s до t в G' .

Задача 7.3 (MAKE CHANGE).

Ресто, но без ограничение за стойностите на монетите.

Вход: $A[1..n]$ - масив от деноминации (рационални числа), W - ресто.

Изход: $\min \{ \sum_{i=1}^n x_i \mid \sum_{i=1}^n x_i A[i] = W \}$.

Нека

$$C[k] = \min \left\{ \sum_{i=1}^n x_i \mid \sum_{i=1}^n x_i A[i] = k \right\}$$

Тогава (докажи) е в сила, че

$$C[k] = \begin{cases} \min_{A[i] \leq k} \{C[k - A[i]] + 1\} & \text{ако } k > 0 \\ 0 & \text{ако } k = 0 \end{cases}$$

Графът

$$G = (\{c_0, c_1, \dots, c_W\}, \{(c_i, c_j) \mid 0 \leq i, j \leq W \text{ \& } j = i + A[t] \text{ за някое } 1 \leq t \leq n\})$$

е ацикличен, т.е. може да приложим техниката на динамичното програмиране. Топологическо сортиране на върховете е например $c_0, c_1, \dots, c_W$.

MAKECHANGE($A[1..n]$: increasing positive integers, $A[1] = 1$, W : positive integer)

```

01   $C[0..W] \leftarrow \text{malloc}(W + 1, \infty)$ : ARRAY of integers
02   $C[0] = 0$ 
03  for  $i = 1$  to  $W$  do
04    for  $j = 1$  to  $n$  do
05      if  $i + A[j] \leq W$  then
06         $C[i + A[j]] \leftarrow \min \{C[i + A[j]], C[i] + 1\}$ 
07      end if
08    end for
09  end for
10  return  $C[W]$ 
```

Времева сложност: $\mathcal{O}(nW)$, което не е полиномиален алгоритъм (сложността е експоненциална спрямо големината на входа).

Алгоритъм, който е полиномиален по стойността на входа, наричаме *псевдополиномиален*.

Задача 7.4 (Брой думи с дължина n в регулярен език). Колко са думите с дължина m над фиксирана Σ , разпознавани от даден автомат?

Вход: $\mathcal{A} = \langle Q, \Sigma, \delta, s, F \rangle$ - представяне на автомат с $|Q| = n$; $m \in \mathbb{N}^+$.

Изход: $|L(\mathcal{A}) \cap \Sigma^m|$

Идея: Опитайте да изчислите

$$a_{q,i} = \left| \left\{ w \mid s \xrightarrow{w} q \text{ \& } |w| = i \right\} \right| \quad \text{за } q \in Q, i \in \{0, \dots, m\}$$

Идея: Постройте матрица A от ред $|Q|$, с комбинаторен смисъл

$$(A^t)_{ij} = \left| \left\{ w \mid i \xrightarrow{w} j \text{ \& } |w| = t \right\} \right|$$

Задача 7.5 (LONGEST COMMON SUBSEQUENCE).

Обща подредица на думите ω_1 и ω_2 наричаме редица от индекси $1 \leq i_1 < i_2 < \dots < i_k \leq \min\{|\omega_1|, |\omega_2|\}$, за която $\omega_{1i_1} = \omega_{2i_1}, \omega_{1i_2} = \omega_{2i_2}, \dots, \omega_{1i_k} = \omega_{2i_k}$.

Вход: $A[1\dots n], B[1\dots m]$ - рационални числа (думи над изброима азбука $\mathbb{Q}$)

Изход: $S[1\dots k]$ - най-дълга обща подредица на A и B .

Дефинираме

$s_{x,y}$ като дължината на най-дълга обща подредица на $A[1\dots x]$ и $B[1\dots y]$ за $0 \leq x \leq n, 0 \leq y \leq m$

Ясно е, че за всяко $0 \leq y \leq m$ е изпълнено, че $s_{0y} = 0$, както и че за всяко $0 \leq x \leq n$ е изпълнено, че $s_{x0} = 0$.

Доказваме, че

$$s_{x,y} = \begin{cases} 0 & \text{ако } x = 0 \vee y = 0 \\ s_{x-1,y-1} + 1 & \text{ако } x, y > 0 \ \& \ A[x] = B[y] \\ \max\{s_{x-1,y}, s_{x,y-1}\} & \text{ако } x, y > 0 \ \& \ A[x] \neq B[y] \end{cases}$$

Търсеният отговор е $a_{x,y}$. Графът, породен от рекурентната зависимост е ацикличен, т.е. можем да приложим техниката на динамичното програмиране.

LCS($A[1\dots n], B[1\dots m]$: arrays of rationals)

```

@1  S[0...n][0...m] ← malloc((n+1)*(m+1)): ARRAY of naturals
@2  for i = 0 to n do S[i][0] = 0
@3  for i = 0 to m do S[0][i] = 0
@4  for i = 1 to n do
@5    for j = 1 to m do
@6      if A[i] = B[j] then
@7        S[i][j] = S[i-1][j-1] + 1
@8      else
@9        S[i][j] = max{S[i-1][j], S[i][j-1]}
@10     end if
@11   end for
@12 end for
@13 {S[n][m] съдържа дължината на максимална обща подредица на A[1...n] и B[1...m]}
@14 C[1...max{n,m}] ← malloc(max{n,m}): ARRAY of rationals
@15 k ← 0, x ← n, y ← m
@16 while x > 0 & y > 0 do
@17   if A[x] = B[y] then
@18     k ← k + 1, C[k] ← A[x], x ← x - 1, y ← y - 1
@19   else if A[x-1][y] > B[x][y-1] then
@20     x ← x - 1
@21   else
@22     y ← y - 1
@23   end if
@24 end while
@25 return REVERSE(C[1...max{n,m}])

```

Коректността на намерена дължина на най-дълга обща подредица на @13 следва от доказаната рекурентна зависимост, а коректността на върнатата подредица може да се докаже с индукция по i и j .

Времевата сложност е $\Theta(nm)$.

Задача 7.6 (LCS PERMUTATIONS).

Вход: $P_1[1...n], P_2[1...n]$ - пермутации π_1 и π_2 на $\{1, \dots, n\}$.

Изход: $A[1...k]$ - най-дълга обща подредица на π_1 и π_2 .

Лема 7.3. За всяка подредица $1 \leq i_1 < i_2 < \dots < i_k \leq n$ е изпълнено, че $\pi_1(i_1), \dots, \pi_1(i_k)$ е обща подредица за π_1 и π_2 т.с.т.к. $\pi_1^{-1}(\pi_2(i_1)) < \pi_1^{-1}(\pi_2(i_2)) < \dots < \pi_1^{-1}(\pi_2(i_k))$ е растяща подредица на $\pi_1^{-1} \circ \pi_2$.

Твърдение 7.4 (редукция на LIS към LCS).

LONGEST INCREASING SUBSEQUENCE $\lll_{n \log n}$ **LONGEST COMMON SUBSEQUENCE**

Свидетел за редукцията е следният алгоритъм, решаващ **LIS**:

1. Сортираме подаденият на вход масив $A[1...n]$ до масив $A'[1...n]$
2. Връщаме резултата от $\text{LCS}(A'[1...n], A[1...n])$

Посоченият алгоритъм е коректен, което се дължи на факта, че

$$\{s \mid s \text{ е обща подредица на } A[1...n] \text{ и } A'[1...n]\} = \{i \mid i \text{ е растяща подредица на } A[1...n]\}$$

Задача 7.7 (Контролно 2, 2022/2023г.).

Интервален граф наричаме граф $G = (V, E)$, за който:

$V = \{[a_i, b_i] \mid a_i, b_i \in \mathbb{Q}\}_{i=1}^n$ е множество от интервали, а $E = \{(I, J) \mid I, J \in V \text{ и } I \cap J \neq \emptyset\}$.

1. Да се докаже, че за всеки интервален граф G с n върха $I_1, \dots, I_n$, има интервален граф $G' = (V', E')$ с n върха $I'_1, \dots, I'_n$ със свойствата:

- краищата на всеки от интервалите I'_j е точка от множеството $\{0, 1, \dots, 2n\}$.
- за всеки два индекса $1 \leq j, k \leq n$ е в сила, че $I_k \cap I_j = \emptyset \iff I'_k \cap I'_j = \emptyset$.

2. Да се докаже, че има алгоритъм със сложност $\mathcal{O}(|V| \log |V|)$, който решава

CANONICAL INTERVALS:

Вход: $I[1...n][1...2]$ - масив от интервали с краища $I[i][1], I[i][2] \in \mathbb{Q}$.

Изход: $I[1...n][1...2]$ - масив от интервали с краища измежду точките $\{0, 1, \dots, 2n\}$, за които $I_k \cap I_j = \emptyset \iff I'_k \cap I'_j = \emptyset$.

3. Нека $\mathcal{I} = \{I_i\}_{i=1}^n$ е множество от интервали, а $c : \mathcal{I} \rightarrow \mathbb{Q}^+$ е неотрицателна функция. За рационални точки $a, b \in \mathbb{Q}$ казваме, че $\pi = (I_{i_0}, \dots, I_{i_k})$ е път от a до b , ако:

$$a \in I_{i_0}, b \in I_{i_k} \text{ и } \forall j < k \ (I_{i_j} \cap I_{i_{j+1}} \neq \emptyset)$$

Цена на пътя π наричаме сумата от стойностите $c(I_{i_j})$ на отделните интервали, през които минава π , тоест:

$$\text{cost}_c(\pi) = \sum_{j=0}^k c(I_{i_j})$$

Да се докаже, че има алгоритъм с времева сложност $\mathcal{O}(n^2)$, който решава **SHORTESTINTERVALPATH**:

Вход: $I[1..n][1..2]$ - масив от интервали с краища $I[i][1], I[i][2] \in \mathbb{Q}$, $c[1..n]$ - масив от рационални числа, като $c[i]$ е стойността на интервала $I[i]$, $a, b \in \mathbb{Q}$.

Изход: $()$, ако няма път от a до b през интервалите $I[1..n]$, иначе $\pi[1..m]$ - път от a до b с възможно най-малка цена според c .

4. Да се докаже, че има алгоритъм с времева сложност $\mathcal{O}(n \log n)$, който решава **SHORTESTINTERVALPATH** при допълнителното предположение, че измежду дадените интервали $I_1, \dots, I_n$ няма такива $i \neq j$, за които $I_i \subseteq I_j$.

Задача 7.8 (MINIMUM MEAN WEIGHT CYCLE).

Нека $G = (V, E)$ е ориентиран граф, $w : E \rightarrow \mathbb{R}$ и $n := |V|$. За цикъл $c = (e_1, e_2, \dots, e_k)$ в G дефинираме

$$\mu(c) = \frac{1}{k} \sum_{i=1}^k w(e_i)$$

Нека $\mu^* = \min \{\mu(c) \mid c \text{ е цикъл в } G\}$.

Нека $\delta : V^2 \rightarrow \mathbb{R} \cup \{-\infty, \infty\}$ и $\delta_k : V^2 \rightarrow \mathbb{R} \cup \{-\infty, \infty\}$ са дефинирани като:

$$\delta(u, v) = \inf \{w(\pi) \mid \pi \text{ е път от } u \text{ до } v \text{ в } G\}$$

$$\delta_k(u, v) = \inf \{w(\pi) \mid \pi \text{ е път от } u \text{ до } v \text{ в } G \text{ и } |\pi| = k\} \quad \text{за } k \in \mathbb{N}$$

Нека БОО $s \in V$ е такъв, че има път от s до v в G за всеки връх $v \in V$. (В противен случай ще разгледаме силно свързаните компоненти на G)

1. Докажете, че ако $\mu^* = 0$, то G не съдържа цикли с отрицателно тегло и

$$\delta(s, v) = \min_{0 \leq k \leq n-1} \delta_k(s, v) \text{ за всеки връх } v \in V$$

Доказателство. Ако c е отрицателен цикъл в G , то $\omega(c) < 0$ и следователно $\mu(c) < 0$, откъдето и $\mu^* \leq \mu(c) < 0$, което е невъзможно.

Нека π е свидетел за $\delta(s, v)$ с най-малка дължина. Ако $|\pi| \geq n$, то π съдържа цикъл c , за който $\omega(c) \geq 0$. Тогава за π' , получен от π след премахване на c , имаме, че $\omega(\pi') \leq \omega(\pi) = \delta(s, v)$, т.е. π' е най-лек и $0 \leq |\pi'| \leq k - 1$. $\square$

2. Докажете, че ако $\mu^* = 0$, то

$$\max_{0 \leq k \leq n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} \geq 0$$

за всеки връх $v \in V$.

Доказателство. Повтаря идеята за път с дължина n , който съдържа цикъл. $\square$

3. Нека c е цикъл, $w(c) = 0$ и u и v са върхове от c . Докажете, че ако пътят по c от u до v е x , то $\delta(s, v) = \delta(s, u) + x$.

Доказателство. Допускаме, че $\delta(s, v) < \delta(s, u) + x$. Пътят от v до u по c е с цена $-x$. Сега имаме, че $\delta(s, v) - x < \delta(s, u)$, което е невъзможно от неравенство на триъгълника. $\square$

4. Докажете, че ако $\mu^* = 0$ и s е цикъл, за който $w(s) = 0$, то има връх v от s , такъв, че

$$\max_{0 \leq k \leq n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} = 0$$

Доказателство. Нека $v \in s$ и π е прост път свидетел за $\delta(s, v)$, $|\pi| = k < n$. Продължаваме π по цикъла s до получаване на π' с дължина n и край v_t . Сега

$$\delta(s, v) + x \geq \delta_n(s, v_t) \geq \delta(s, v_t) = \delta(s, v) + x \quad (\text{от 3})$$

Тоест $\delta_n(s, v_t) = \delta(s, v_t)$. Сега за $v_t \in c$:

$$0 \leq \max_{0 \leq k \leq n-1} \frac{\delta_n(s, v_t) - \delta_k(s, v_t)}{n - k} = \max_{0 \leq k \leq n-1} \frac{\delta(s, v_t) - \delta_k(s, v_t)}{n - k} \leq 0$$

(за всяко k и v имаме $\delta(s, v) \leq \delta_k(s, v)$)

т.е.

$$\max_{0 \leq k \leq n-1} \frac{\delta_n(s, v_t) - \delta_k(s, v_t)}{n - k} = 0$$

□

5. Докажете, че ако $\mu^* = 0$, то

$$\min_{v \in V} \max_{0 \leq k \leq n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} = 0$$

6. Докажете, че ако $\nu(e) = \mu(e) + t$, то $\nu^* = \mu^* + t$.

Заклучете, че

$$\mu^* = \min_{v \in V} \max_{0 \leq k \leq n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k}$$

7. Да се предложи алгоритъм с времева сложност $O(VE)$, решаващ проблема MINIMUMMEANWEIGHTCYCLE:

Вход: $V[1...n], E[1...m], M[1...m]$ - представяне на функция $\mu: E \rightarrow \mathbb{Q}$.

Изход: μ^*

Задача 7.9 (BITONIC EUCLIDEAN TRAVELING SALESMAN).

Разглеждаме специален случай на задачата за търговския пътник. Градовете, които трябва да посети и да се върне в началния са точки в равнината с различни абсциси, а разстоянието между произволни две точки е евклидово. Цикълът s :

$$(x_0, y_0), (x_1, y_1), \dots, (x_k, y_k), (x_0, y_0)$$

наричаме **битоничен**, ако има $0 \leq t \leq k$, за което

$$x_0 < x_1 < \dots < x_t > x_{t+1} > \dots > x_k > x_0$$

или се свежда до такъв след смяна на началната и крайна точка. Цената $\nu(s)$ наричаме сумата от евклидовите разстояния между съседни точки.

Нека $\nu^*((x_0, y_0), (x_1, y_1), \dots, (x_n, y_n))$ е цената на най-евтин битоничен цикъл, в който участват (x_i, y_i) за $0 \leq i \leq n$.

Вход: $A[1...n], B[1...n]$ - представяне на точки в равнината.

Изход: $\nu^*((A[1], B[1]), \dots, (A[n], B[n]))$

Ясно е, че за такъв битоничен цикъл индексът p ще бъде индексът на точката с най-голяма абсциса, т.е. "Хамилтоновият" цикъл ще започва от точката с най-малка абсциса, върви надясно до тази с най-малка абсциса, след което се връща по останалите точки до началната (тази с най-малка абсциса).

Нека след сортиране по абсциса за $\mathcal{O}(n \log n)$ време разполагаме с точките

$$(A'[1], B'[1]), \dots, (A'[n], B'[n])$$

Опитваме са да пресметнем количествата $a_{i,j}$ за $1 \leq i < j \leq n$, така, че $a_{i,j}$ да бъде сумарно най-леката цена на два пътя π_1, π_2 с начало $(A'[1], B'[1])$, такива, че:

- $(A'[1], B'[1])$ е единственият общ връх на π_1 и π_2
- върховете на двата пътя са наредени нарастващо по абсциса
- обединението на върховете на π_1 и π_2 е $\{(A'[1], B'[1]), \dots, (A'[j], B'[j])\}$
- край е π_1 е $(A'[i], B'[i])$, а на π_2 е $(A'[j], B'[j])$

Тогава $\nu^* = a_{n-1,n} + d((A'[n-1], B'[n-1]), (A'[n], B'[n]))$. (Защо?)

Да се докаже, че е в сила:

$$a_{i,j} = \begin{cases} d((A'[1], B'[1]), (A'[2], B'[2])) & \text{ако } i = 1, j = 2 \\ a_{i,j-1} + d((A'[j-1], B'[j-1]), (A'[j], B'[j])) & \text{ако } i < j - 1 \\ \min_{1 \leq k \leq j-2} a_{k,j-1} + d((A'[k], B'[k]), (A'[j], B'[j])) & \text{ако } i = j - 1 \end{cases}$$

Сега числата $a_{i,j}$ можем да пресметнем по схемата за динамично програмиране. Ако G е графът, породен от горната зависимост, то

$$|V| = n(n-1)/2$$

$$|E| = n(n-1)/2 + n(n-1)/2 - (n-1)$$

т.е. $\mathcal{O}(|V| + |E|) = \mathcal{O}(n^2)$ е сложността на получения алгоритъм.

BITONICETSP($A[1..n], B[1..n]$: points representation)

```

01  SORT( $A[1..n], B[1..n]$ )
02   $P \leftarrow \text{malloc}(n * n)$ : ARRAY of rationals
03   $P[1][2] \leftarrow d((A[1], B[1]), (A[2], B[2]))$ 
04  for  $j = 3$  to  $n$  do
05    for  $i = 1$  to  $j - 2$  do
06       $P[i][j] \leftarrow P[i][j-1] + d((A[j-1], B[j-1]), (A[j], B[j]))$ 
07    end for
08     $P[j-1][j] \leftarrow \infty$ 
09    for  $k = 1$  to  $j - 2$  do
10       $P[j-1][j] \leftarrow \min \{P[j-1][j], P[k, j-1] + d((A[k], B[k]), (A[j], B[j]))\}$ 
11    end for
12  end for
13  return  $P[n-1][n] + d((A[n], B[n]), (A[n], B[n]))$ 

```

8 P, NP, NP-HARD, NP-COMPLETE

Интересуваме се само от разпознавателни проблеми и за тях ще говорим като за езици. Проблемът дефиниран като Вход/Изход, считаме за множество от кодирания на входовете и изходите, конкатенирани.

Определение 8.1 (класът проблем P). P е класът от онези езици L , за които има детерминирана МТ $\mathcal{M}$, че:

- $L(\mathcal{M}) = L$;
- има полином p , такъв че времевата сложност в най-лошия случай на $\mathcal{M}$ е $\mathcal{O}(p(n))$ за всяко n .

Това са езиците, за които имаме детерминиран полиномиален алгоритъм.

Определение 8.2 (класът проблеми NP). NP е класът от онези езици L , за които има недетерминирана МТ $\mathcal{M}$, че:

- $L(\mathcal{M}) = L$;
- има полином p , такъв че $\forall n \in \mathbb{N} (Time_{\mathcal{M}}(n) \text{ е дефинирано} \implies Time_{\mathcal{M}}(n) \leq p(n))$.

Това са проблеми, за които ако имаме сертификат (кандидат за свидетел), можем за полиномиално да проверим дали той наистина е свидетел за вярност.

Определение 8.3 (Редукция за полиномиално време). Казваме, че език L се свежда за полиномиално време към език L' и пишем $L \leq_p L'$, ако има функция f :

1. $f(w)$ може да бъде изчислена за полиномиално време от ДМТ;
2. $w \in L \iff f(w) \in L'$

По този начин построяваме ДМТ за L , разчитайки на наличието на "черна кутия", ДМТ за L' .

Определение 8.4 (класът проблеми NP-HARD). Казваме, че езикът L е $NP - hard$, ако

$$\forall L_0 \in NP (L_0 \leq_p L)$$

Проблеми, които са "по-трудни" от всички NP проблеми.

Определение 8.5 (класът проблеми NP-COMPLETE). Езикът L е $NP - complete$, ако:

1. $L \in NP$;
2. $L \in NP - hard$.

Твърдение 8.1. Ако $L \in NP-HARD$ и $L \leq_p L'$, то $L' \in NP-HARD$.

Отворен е въпросът дали $P=NP$, което е еквивалентно с $P \cap NP-HARD \neq \emptyset$.

Ако разполагаме с $NP-HARD$ проблем L и целим да докажем, че $L' \in NP-HARD$, то е достатъчно да покажем, че $L \leq_p L'$, т.е. да модифицираме за полиномиално време произволна инстанция на проблема L .

Твърдение 8.2 (TRAVELING SALESMAN е NP-COMPLETE).

Вход: $G(V, E)$ - граф ($E = V \times V$), $c: E \rightarrow \mathbb{Q}$, $k \in \mathbb{Q}$

Изход: Има ли Хамилтонов цикъл π в G , такъв че $c(\pi) \leq k$?

Частен случай на проблема има полиномиално решение, разгледано [тук](#).

Ще покажем, че

$$\text{HAMILTONIAN CYCLE} \leq_p \text{TRAVELING SALESMAN}$$

Доказателство. Проблемът се разпознава от НДМТ, генерираща недетерминирано всички редици от върхове с дължина n , след което за полиномиално време проверява дали редицата е Хамилтонов цикъл и дали цената му е $\leq k$, т.е. проблемът е NP.

Нека $G(V, E)$ е инстанция на **HAMILTONIAN CYCLE**.

Нека $G' = (V, V \times V)$ и $c: V \times V \rightarrow \mathbb{Q}$ е дефинирана като:

$$c(u, v) = \begin{cases} 1, & \text{ако } (u, v) \in E \\ 2, & \text{иначе} \end{cases}$$

Нека $k = |V|$. Остава да покажем, че $G(V, E)$ има Хамилтонов път т.с.т.к. G' има Хамилтонов цикъл с цена $\leq k$. □

Задача 8.1 (INTEGER (LINEAR) PROGRAMMING).

Задачата на линейното програмиране има известни полиномиални алгоритми (като например методът на елипсоидите, симплекс методът има експоненциална сложност в най-лошия случай). Когато обаче ограниченията изискват само цели числа, задачата става NP-трудна.

Вход: $A[1...m][1...k], b[1...m]$ - ограничения (реални числа), $c[1...k]$ - реални числа, v - реално число.

Изход: Има ли $x \in \mathbb{Z}^k$, за който $Ax^T \leq b^T$ и $cx^T \geq v$?

Твърдение 8.3 (IP ∈ NP-HARD (редукция от 3SAT)).

$$3\text{SAT} \leq_p \text{IP}$$

Доказателство. Защо е NP?

Нека $\varphi = C_1 \& C_2 \& \dots \& C_k$, където $C_i = l_{i1} \vee l_{i2} \vee l_{i3}$ (l_{ij} е литерал) е екземпляр на **3SAT**.

Нека

$$v(l_{i,j}) = \begin{cases} x & \text{ако } l_{ij} = x \text{ за някое } x \in \text{Var}(\varphi) \\ (1 - x) & \text{ако } l_{ij} = \bar{x} \text{ за някое } x \in \text{Var}(\varphi) \end{cases}$$

Разглеждаме системата с коефициенти A и десни страни b (променливи са $\text{Var}(\varphi)$):

$$\begin{aligned} 0 \leq x \leq 1 & \text{ за всяко } x \in \text{Var}(\varphi) \\ v(l_{i1}) + v(l_{i2}) + v(l_{i3}) & \geq 1 \text{ за всяко } 1 \leq i \leq k \end{aligned}$$

Нека $c = (1, \dots, 1)$.

Сега φ е изпълнима т.с.т.к. има $x \in \mathbb{Z}^{|\text{Var}(\varphi)|}$, че $Ax^T \leq b$ и $cx^T \geq 0$ (т.с.т.к. има $x \in \mathbb{Z}^{|\text{Var}(\varphi)|}$, че $Ax^T \leq b$). □

Твърдение 8.4 ($\text{IP} \in \text{NP-HARD}$ (редукция от **VERTEX COVER**)).

$$\text{VERTEX COVER} \leq_p \text{IP}$$

Доказателство. Нека $G = (V, E)$ е неориентиран граф и $r \in \mathbb{R}$ (екземпляр на **VERTEX COVER**). Разглеждаме системата с коефициенти A и десни страни b (неизвестни са x_v за всяко $v \in V$):

$$0 \leq x_u + x_v \leq 1 \text{ за всяко } \{u, v\} \in E$$

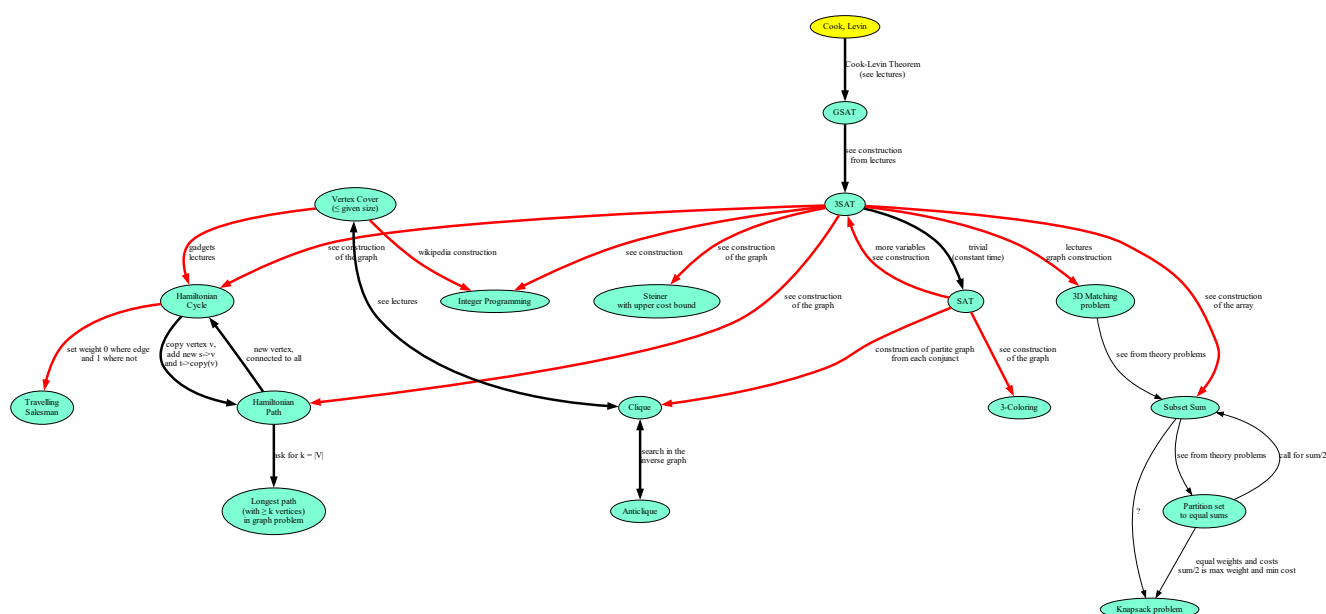
Нека $c = (-1, \dots, -1)$.

Сега G има върхово покритие с $\leq r$ върха т.с.т.к. има $x \in \mathbb{Z}^{|V|}$, че $Ax^T \leq b$ и $cx^T \geq -r$

□

Допълнение 8.1 (Някои редукции за полиномиално време).

Свали с линкове от [тук](#)



9 Задачи, неразгледани на упражнение

Задача 9.1 (HOLE). Даден е масив с n различни числа от 1 до $n + 1$. Да се намери първото положително липсващо число в масива.

Вход: $A[1...n]$, $A[i] \leq n + 1$, $1 \leq i \leq n$. - масив от пол. естествени числа.

Изход: $\min \{i \mid i \in \mathbb{N} \ \& \ i \notin A\}$.

Задача 9.2 (Домашно 1, Контролно 1, 2023г.).

1. На окръжност по часовниковата стрелка, са написани n реални числа, $a_0, \dots, a_{n-1}$, със сума 0. Да се докаже, че има $i \leq n$, за което:

$$\sum_{k=0}^j a_{(i+k) \bmod n} \geq 0 \text{ за всяко } j \geq 0. \quad (\star)$$

2. Да е предложи алгоритъм със сложност $\mathcal{O}(n)$, който решава следния проблем:

Вход: $A[0...n-1]$ - масив от цели числа със сума 0.

Изход: i , за което е изпълнено $(\star)$.

3. Да се докаже, че за всяко непразно множество $M = \{x_1, \dots, x_n\}$ от точки в равнината и всяка права l има точка $x_i \in M$, такава, че правата през x_i , успоредна на l , разделя равнината на две полуравнини, като една от двете не съдържа точки от M

4. Да се предложи алгоритъм със сложност $\mathcal{O}(n)$, който решава следния проблем:

Вход: $M[1...n][2]$ - масив от целочислени точки в равнината, $v[2]$ - ненулев целочислен вектор

Изход: i - индекс на точка $M[i]$, такава, че $\langle \overrightarrow{M[i]M[j]}, v \rangle \geq 0$ за всяко $j \leq n$.

Задача 9.3 (Алгоритъм на Boyer – Moore).

Изборните резултати са представени с масив от вотове $V[1...n]$, като $V[i]$ е подаден вот за кандидат i (броят на кандидатите не ни е известен). Изборите се печелят от кандидат, когато гласовете за него са над 50%, т.е. поне $\lfloor \frac{n}{2} \rfloor + 1$.

Да се предложи алгоритъм с времева сложност $\Theta(n)$, който решава следния проблем:

Вход: $V[1...n]$ - масив от вотове (положителни естествени числа).

Изход: i - победител в изборите, ако има такъв, в противен случай - 0.

Задача 9.4 (MAX CLUSTER).

Вход: $A[1...n]$ - масив от рационални числа, $d \in \mathbb{Q}^+$.

Изход: $\max \{|S| \mid S \subseteq A \ \& \ \forall x, y \in S (|x - y| \leq d)\}$.

Задача 9.5 (INTERSECTING SEGMENTS).

Разглеждаме правите в равнината $y = 0$ и $y = 1$. Върху $y = 0$ са разположени n различни точки

$$(a_1, 0), (a_2, 0), \dots, (a_n, 0)$$

Върху $y = 1$ са разположени n различни точки

$$(b_1, 1), (b_2, 1), \dots, (b_n, 1)$$

С s_i бележим отсечката с краища $(a_i, 0)$ и $(b_i, 1)$.

Вход: $A[1..n], B[1..n]$ - масив от абсцисите на точките.

Изход: $|\{(s_i, s_j) \mid 1 \leq i < j \leq n \text{ \& } s_i \cap s_j \neq \emptyset\}|$.

Задача 9.6 (LOCALMIN).

Ще казваме, че индексът $1 < i < n$ е локален минимум за масива $A[1..n]$, когато за всеки индекс $1 \leq j \leq n$

$$\text{ако } |i - j| = 1, \text{ то } A[i] < A[j]$$

Предложете алгоритъм с времева сложност $O(\log n)$, решаващ следния проблем:

Вход: $A[1..n]$ - масив от рационални числа, в който има индекс на локален минимум

Изход: i - индекс на локален минимум.

Задача 9.7 (TYPESETTING).

При писане на голям текст в документ е неизбежно в края на някои редове да остава неизползвано "празно" пространство. С подходящо разделяне на текста по редове целим да минимизираме сумарното празно пространство по всички редове.

Текст моделираме с редица от думи $\{\omega_i\}_{i=1}^n$, като между две последователни думи, записани на един и същ ред, стои точно един символ $_$. Известно е, че на един ред могат да се запишат най-много L символа.

Запис на даден текст $\{\omega_i\}_{i=1}^n$ наричаме растяща редица от индекси $0 = l_0 \leq l_1 \leq \dots \leq l_k = n$, като k е броят редове, на които е записан текстът.

Сумарното празно пространство, породено от запис на текст ще бележим с

$$p(l) = \sum_{i=1}^k \left(L - (l_i - l_{i-1}) - \sum_{j=l_{i-1}+1}^{l_i} |\omega_j| \right)$$

ако за всяко $1 \leq i \leq k$ е изпълнено, че $L - (l_i - l_{i-1}) - \sum_{j=l_{i-1}+1}^{l_i} |\omega_j| \geq 0$, иначе $p(l) = \infty$.

Решаваме следния проблем:

Вход: $W[1..n]$ - представяне на текст като масив от думи.

Изход: $\min \{p(l) \mid l \text{ запис на } W\}$

Задача 9.8 (STABLE MATCHING).

"Стабилно съчетание"

Класически контекст с мъже и жени

Съчетание наричаме биекция $I_n \rightarrow I_n$.

Нека $(i, p(i))$ и $(j, p(j))$ са партньори, определени от p . Двойката от мъж и жена (i, j) е нестабилна в p , ако

$$m_i(p(j)) > m_i(p(i)) \text{ и } m_j(p(i)) > m_j(p(j))$$

Съчетание p наричаме стабилно съчетание, ако няма нестабилни двойки в p .

Вход: $M[1..n][1..n], F[1..n][1..n]$ - матрици, представящи предпочитанията на мъжете и жените

към другия пол

Изход: $P[1...n]$ - представяне на стабилно съчетание, където мъжът i е партньор на жената $P[i]$.

Задача 9.9 (CAMPAIGNING).

В надпреварата за местните избори участват кандидатите **A** и **B**. В продължение на n дни, в един град на ден те се борят за гласовете на електората в n различни града, номерирани с 1 до n . Известно е, че гласоподавателите в град i са d_i , т.е. гласовете, за които се борят, са $D = \sum_{i=1}^n d_i$.

За победа на изборите са нужни поне $\lfloor \frac{D}{2} \rfloor + 1$ гласа.

Кандидат **A** вече е направил проучване и очакваният брой гласове, които ще получи в град i , е e_i . За съжаление $\sum_{i=1}^n e_i < \lfloor \frac{D}{2} \rfloor + 1$, т.е. към момента ще загуби изборите.

A обаче има екип от агитатори, които могат да спечелят всичките d_i гласове в град i , след което се нуждаят от два дни почивка, в които да преместят екипа в друг град. Предложете $O(n)$ алгоритъм, който решава следния проблем:

Вход: $D[1...n]$ - брой гласоподаватели, $E[1...n]$ - очаквани гласове за **A**.

Изход: $A[1...k]$ - градовете, в които екипът ще агитира за **A**, че да спечели, ако е възможно, иначе - -1 .

Задача 9.10 (TRAINING HILLS).

Лятото идва и бегачът Ради си е наумил, че трябва отново да влезе в добра физическа форма. Той е решил, че най-ефективно за него ще бъде да прави тренировки из града си, в който всички шосета между две кръстовища са отсечки в пространството. Градът му е пълен с нагорница и надолница и Ради е развил лош навик, че веднъж намалил надморската височина на която се намира, го домързява да се изкачва отново. **Поради същата причина Ради се стреми и разстоянието, което ще избяга, да е минимално.** Планът му се състои единствено от начално и крайно кръстовище.

Той разполага с карта, на която са отбелязани кръстовищата $1...n$, като кръстовище i е с координати $\langle x_i, y_i, z_i \rangle$ е на надморска височина z_i .

Формално, тренировка за Ради от s до t е редица от кръстовища $x_1, \dots, x_k$, такива, че няма индекс $1 < i < k$, такъв, че $z_{x_{i-1}} > z_{x_i} < z_{x_{i+1}}$.

Желана тренировка за Ради от s до t е тренировка от s до t с **минимална** сума от дължините на улиците, по които минава.

1. Да се докаже, че има алгоритъм с времева сложност $O(m+n \log n)$, който решава проблема TRAINING:

Вход: $A[1...n]$ - списък от кръстовища в града като наредени тройки $\langle x_i, y_i, z_i \rangle$, $B[1...m]$ - списък от улици като наредени двойки $\langle i, j \rangle$ s, t - индекси на съответно начално и крайно кръстовище.

Изход: $T[1...k]$ - желана тренировка за Ради от s до t .

2. Градът на Ради се оказва толкова хълмист, че няма две свързани кръстовища на една и съща надморска височина.

Да се докаже, че има алгоритъм с времева сложност $O(m+n)$, който решава проблема TRAININGHILLS:

Вход: $V[1...n], E[1...m]$ - представяне на града на Ради. $C[1...m]$ - представяне на инекция $c: E \rightarrow \mathbb{Q}$, $A[1...n]$ - надморски височини на кръстовищата, s, t - съответно начално

и крайно кръстовище.

Изход: $T[1...k]$ - желана тренировка за Ради от s до t .

Благодарности:

Записките са в този вид с основната подкрепа и съавторство на **Румен Михов**.

Корекции направиха:

- *Никола Драгнев*

Следва продължение...